PROJECT REPORT

on

FUSE-BASED FILE SYSTEM WITH CUSTOM DISK SCHEDULING AND SYNCHRONIZATION



Operating Systems Lab (24B11CS312)

Project Based Learning (PBL)

Odd Semester 2026, Academic Session 2026–27

SUBMITTED TO

Dr. Tanvi Gautam

Dr. Sayani Ghosal

Mr. Umar Farooq

SUBMITTED BY

Saanvi Grover	2408030010
Prankit Goel	2408030011
Aditya Sharma	2408030019
Aaradhya Kulashri	2408030026

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY,

NOIDA (U.P.) NOVEMBER 2026

Table of Contents

1. TITLE OF THE PROJECT **1**

2. ABSTRACT OF THE PROJECT **1**

3. SCOPE OF THE PROJECT **2**

3(a) Functional Requirements 3

3(b) Non-Functional Requirements 4

3(c) Description of the Modules of the Project 5

4. TOOLS AND TECHNOLOGIES USED **12**

5. DESIGN OF THE PROJECT **14**

5.1 System Architecture 14

5.2 System Workflow 15

5.3 Module-Wise Design 17

5.4 Module Interaction 24

5.5 Operating Systems Concepts 25

5.6 Data Flow 28

5.7 Design Decisions 29

6. IMPLEMENTATION DETAILS **30**

6.1 Implementation Overview 30

6.2 Module-Wise Implementation 31

6.3 Source Code 32

6.4 Code Contribution Summary 42

6.5 Implementation Screenshots 43

7. TESTING DETAILS **50**

7.1 Testing Methodology 50

7.2 Test Cases 50

7.3 Testing Results 52

7.4 Testing Screenshots 57

8. REFERENCES **61**

A. SOURCE LISTINGS OF THE SCHEDULING PATH **63**

B. REPRODUCING THE RESULTS **69**

List of Figures

Fig. 1 System architecture of SchedFS. Shaded blue layers are where scheduling takes place. . . . 14
Fig. 2 Component architecture: dependencies between source modules (dashed: data files). . . . 15
Fig. 3 Overall workflow from build to unmount. . . . 16
Fig. 4 Mount sequence. The dispatcher thread is created in `op_init`, after the fork. . . . 17
Fig. 5 On-disk layout of a 16 MiB SchedFS image (widths not to scale). . . . 17
Fig. 6 Inode structure and indexed allocation (12 direct pointers + 1 single-indirect block). . . . 18
Fig. 7 Flowchart of the image formatter (`mkfs/mkfs.c`). . . . 19
Fig. 8 Path resolution. . . . 20
Fig. 9 Write path through M3 and M4. . . . 20
Fig. 10 Hint-based, two-pass block allocation. . . . 21
Fig. 11 Producer side (left) and consumer side (right) of the bounded request queue. . . . 22
Fig. 12 Decision logic of `scheduler_pick()` for the four policies. . . . 23
Fig. 13 Dispatcher service loop (left) and benchmark replay loop (right). . . . 24
Fig. 14 Sequence diagram of one scheduled block I/O. . . . 25
Fig. 15 Head movement for the textbook trace. SCAN first moves towards block 0 and reaches the edge before reversing. . . . 26
Fig. 16 Life cycle of a disk request. . . . 27
Fig. 17 Global lock-acquisition order used to prevent deadlock. . . . 28
Fig. 18 Level-1 data-flow diagram. . . . 29
Fig. 19 Building the three executables with strict warning flags; the compiler emits no warnings. . . . 44
Fig. 20 Formatting an image, refusing to overwrite it, rejecting invalid sizes, and dumping the superblock. . . . 44
Fig. 21 Mounting under SSTF and using the file system with ordinary commands. 45
Fig. 22 Truncate, rename, capacity reporting and error handling on the mounted file system. . . . 46
Fig. 23 Unmount summary in `filesystem.log`, and data persisting after a remount under FCFS. . . . 46

Fig. 24 Verbose mode under SCAN: DEBUG entries show five consecutive data blocks allocated for a 20,000-byte file. 47
Fig. 25 Interactive demo page (GitHub Pages): overview of the project. 47
Fig. 26 Demo page: the request path and the on-disk layout, linked to the source files. 48
Fig. 27 Scheduler lab after playing the textbook trace: FCFS 640, SSTF 236, SCAN 236 and Adaptive 299 blocks. 48
Fig. 28 Scheduler lab on the 16-request concurrent trace: FCFS zig-zags across the disk (27,402 blocks). 49
Fig. 29 Repository on GitHub, with the merged pull requests and the `github-pages` deployment. 49
Fig. 30 Total seek distance per workload and policy (lower is better). 54
Fig. 31 Average waiting time per workload and policy (lower is better). 55
Fig. 32 Mean total seek on the mounted file system; error bars show the minimum and maximum of three trials. 56
Fig. 33 `make test`: scheduler unit test, core persistence and concurrency test, and mounted system tests under all four policies, all passing (TC01–TC03, TC19, TC21, TC22). 57
Fig. 34 Error handling at mount time: unknown policy, missing image, non-SchedFS image and negative weight (TC16, TC17). 57
Fig. 35 ThreadSanitizer build of the core concurrency test: no data races reported, exit status 0 (TC20). 58
Fig. 36 Benchmark run and its CSV output (TC24; data in Table 19). 58
Fig. 37 The same mounted workload under each policy, three trials each (TC25; data in Table 20). 59
Fig. 38 Benchmark view of the demo page, drawn from the repository's committed `benchmark.csv`. 59
Fig. 39 Summary graph from the plotting script (committed CSV) 60

List of Tables

Table 1 Project identification . . . 1
Table 2 Functional requirements . . . 3
Table 3 Non-functional requirements . . . 4
Table 4 Module overview . . . 5
Table 5 Synopsis modules and where they are implemented . . . 6
Table 6 Synopsis functional requirements and the corresponding requirements in this report . . . 7
Table 7 Tools and technologies . . . 12
Table 8 Superblock of a freshly formatted 16 MiB image (values from `od`, Fig. 20) 18
Table 9 Interfaces between modules . . . 25
Table 10 Scheduling policies implemented in `scheduler.c` . . . 26
Table 11 Design decisions and trade-offs . . . 29
Table 12 Source organisation . . . 30
Table 13 Principal data structures . . . 31
Table 14 Qualifying lines of code per file . . . 42
Table 15 Code contribution summary . . . 43
Table 16 Testing levels and tools . . . 50
Table 17 Test cases and results . . . 51
Table 18 Summary of test results . . . 53
Table 19 Benchmark results (Linux run, Fig. 36); best seek per workload in bold 54
Table 20 Total head movement on the mounted file system (4,927 requests per run) . . . 56

List of Listings

1 On-disk structures with compile-time size checks (`include/layout.h`) . . 33
2 Layout computation and safe creation of the image (`mkfs/mkfs.c`) 33
3 Starting the file system inside the FUSE daemon (`src/fuse/fuse_main.c`) 34
4 Inode update protected by the inode-table mutex (`src/fs/filesystem.c`) 34
5 Logical-to-physical block mapping (`filesystem.c`) . . . 34
6 Rename with roll-back of completed steps on failure (`src/fs/filesystem.c`) 35
7 Completion wait and bounded queue (`request_queue.c`) . . . 36
8 Request selection for all four policies (`src/sched/scheduler.c`) 37
9 Dispatcher thread and synchronous transfer (`virtual_disk.c`) 39

10 Thread-safe metric accumulation (`src/bench/metrics.c`) 40
11 Replay loop and SCAN edge accounting (`src/bench/benchmark_main.c`) 40
12 Textbook-trace unit test (`tests/test_scheduler.c`) 41
13 Background-mount regression test (`tests/system.sh`) 42
14 `src/sync/request_queue.c` (complete) 63
15 `src/sched/scheduler.c` (complete) 65
16 `src/storage/virtual_disk.c` (complete) 67
17 Commands used to produce the results 69

1. TITLE OF THE PROJECT

FUSE-Based File System with Custom Disk Scheduling and Synchronization

This is the title approved in the project synopsis. The implementation is called **SchedFS**: a mountable Linux file system written in C11 on the FUSE 3 interface. Every block that the file system reads or writes is turned into a request, queued, and served in the order chosen by a disk-scheduling policy (FCFS, SSTF, SCAN or Adaptive) that is selected when the file system is mounted. The project therefore joins two chapters of an operating-systems course—*file systems* and *disk scheduling*—and adds a third, *process synchronization*, because the queue between them is a bounded producer–consumer buffer shared by several threads.

Table 1 Project identification

Project title	FUSE-Based File System with Custom Disk Scheduling and Synchronization
Implementation	SchedFS (repository name `FUSE`)
Course	Operating Systems Lab (24B11CS312), Project Based Learning
Session	Odd Semester 2026, Academic Session 2026–27
Supervisors	Dr. Tanvi Gautam, Dr. Sayani Ghosal, Mr. Umar Farooq
Category	Systems programming: file systems, disk scheduling, synchronization
Platform	Linux (Ubuntu 22.04/24.04), FUSE 3, GNU C11, POSIX threads
Source repository	`https://github.com/Sharmaditya2k05/FUSE`
Interactive demo	`https://sharmaditya2k05.github.io/FUSE/`

2. ABSTRACT OF THE PROJECT

Operating-systems courses treat file systems and disk scheduling as separate topics, so students rarely see a scheduling policy act on the I/O stream of a real file system. SchedFS joins the two. It is a mountable Linux file system implemented in C11 on the FUSE 3 interface. Its data lives in a single image file with a superblock, a free-block bitmap, 256 fixed-size inodes and 4 KiB data blocks reached through direct and single-indirect pointers. Every block read or write issued by the file system becomes a request in a bounded producer–consumer queue protected by a mutex and two condition variables. A single dispatcher thread selects the next request with a policy chosen at mount time—first-come first-served (FCFS), shortest seek time first (SSTF), SCAN, or an age-aware Adaptive policy—and only then calls `pread` or `pwrite` on the image, while each caller sleeps on its

own condition variable until its request completes. The system also records seek distance and waiting time, and includes a trace-replay benchmark and a web visualisation. On the classic textbook trace the scheduler reproduces head movements of 640, 236 and 236 blocks for FCFS, SSTF and SCAN. On the mounted file system with four concurrent writers, SSTF reduced total head movement by 40 % compared with FCFS. Scheduler unit tests, core concurrency tests and mounted FUSE system tests all pass, and ThreadSanitizer reports no data races.

3. SCOPE OF THE PROJECT

Problem and objectives. Disk-scheduling algorithms are normally studied on hand-written request lists [1, 2], detached from the software that produces disk traffic. The objective of SchedFS is to place these algorithms inside a working file system, so that ordinary commands such as `cp`, `cat` and `mkdir` generate the requests and the effect of each policy can be observed and measured. A second objective is to show correct multi-threaded synchronization: many FUSE worker threads produce requests concurrently, and a single consumer must serve them without races, lost wake-ups or deadlock.

Scope and boundaries. SchedFS implements an on-disk format, a formatter (`mkfs.schedfs`), the core file operations (create, read, write, truncate, rename, unlink, mkdir, rmdir, readdir, stat and statfs), four scheduling policies, metric collection, a benchmark harness and a test suite. Seek distance is modelled as the absolute difference between block numbers, as in textbook disk scheduling; the image file itself is stored on whatever device hosts it, so the project measures scheduling decisions rather than physical seek time. Journaling, crash recovery, hard and symbolic links, extended attributes, ACL enforcement and a block cache are outside the scope.

Intended users. The primary users are operating-systems students and instructors who want to see scheduling and synchronization at work on real I/O. The benchmark and the web visualisation also serve as teaching aids during lectures and viva examinations.

Operating-systems relevance. The project exercises file-system layout and indexed allocation, free-space management with a bitmap, disk scheduling, the bounded-buffer problem, mutexes, condition variables and reader–writer locks, lock ordering for deadlock avoidance, user-space file systems and the system-call interface (`open`, `pread`, `pwrite`, `fsync`, `fork` during daemonization) [1, 3, 4].

Practical applications. The same layering—a request queue in front of a block device with a pluggable policy—is used by real I/O schedulers in operating-system kernels [5]. SchedFS can be used to compare policies on custom workloads, to demonstrate starvation

and fairness, and as a starting point for adding policies such as C-SCAN or LOOK.

Limitations. The maximum file size is 1,036 blocks (about 4.05 MiB). A clean unmount is required for persistence because there is no journal. `rename` refuses to replace an existing target. The image format is native little-endian and intended for Linux on x86-64.

3(a) Functional Requirements

Table 2 lists twelve functional requirements. Each one is implemented and is exercised by a test or a captured run in Section 7.

Table 2 Functional requirements

ID	Requirement	Description and expected functionality	Module
FR1	Format an image	`mkfs.schedfs IMAGE SIZE` creates a new image, writes the superblock, bitmap, inode table and root directory, and refuses to overwrite an existing file or accept an invalid size.	M1 Formatter
FR2	Mount as a Linux file system	`schedfs IMAGE MOUNTPOINT --sched=POLICY` validates the image, prints its geometry and mounts it through FUSE, in the foreground (`-f`) or as a background daemon.	M2 FUSE adapter
FR3	Create, read and write files	Regular files can be created, written at any offset and read back byte-for-byte, including files larger than 48 KiB that need the single-indirect block.	M3, M4
FR4	Directory operations	`mkdir`, `rmdir` and `readdir` work on nested directories; removing a non-empty directory fails with `ENOTEMPTY`.	M3 Inode and directory layer
FR5	Rename, delete and truncate	`rename`, `unlink` and `truncate` are supported; truncating to a larger size creates a sparse region that reads as zeros.	M3, M4
FR6	Metadata and capacity	`stat` reports size, mode, owner and timestamps; `df` reports total and free capacity through `statfs`.	M2, M3
FR7	Scheduled block I/O	Every block read or write after mounting is submitted as a request to a bounded queue and performed only by the dispatcher thread.	M5, M7
FR8	Selectable scheduling policies	FCFS, SSTF, SCAN and Adaptive are selectable at mount time; Adaptive is tunable with `--alpha` and `--beta`. Invalid policy names or negative weights are rejected.	M6 Scheduler
FR9	Persistence	Data and metadata written before a clean unmount are present after remounting, including under a different policy.	M3, M7

ID	Requirement	Description and expected functionality	Module
FR10	Metrics and logging	Request count and total seek distance are logged at unmount; `--verbose` adds DEBUG entries such as block allocations to `filesystem.log`.	M8 Metrics and logging
FR11	Benchmark and graphs	`schedfs-bench --all` replays five fixed traces under every policy and writes `benchmark.csv`; `plot\_results.py` draws comparison graphs.	M9 Benchmark
FR12	Interactive visualisation	A web page animates the head movement of each policy on the textbook and benchmark traces using a port of `scheduler\_pick()`.	M9 Benchmark and visualisation

3(b) Non-Functional Requirements

Table 3 lists eight non-functional requirements, each with the criterion used to check it.

Table 3 Non-functional requirements

ID	Quality	How it applies to SchedFS	Measurable criterion
NFR1	Correctness and reliability	Data written through the mount must read back identically and survive remounting.	`cmp` reports identical files; all `make test` stages pass (Section 7.3).
NFR2	Thread safety	Concurrent FUSE worker threads share the queue, bitmap, inode table and superblock.	ThreadSanitizer reports no data race in the 4-thread core test.
NFR3	Deadlock freedom	Locks are always acquired in one documented global order.	No hang in any test run; lock order in Fig. 17.
NFR4	Determinism	The same request set must give the same service order for a policy.	Ties broken by arrival time, then request ID; FCFS, SSTF and SCAN seek totals are identical across runs.
NFR5	Bounded resource use	The queue holds at most 128 requests; metadata overhead is fixed.	Queue capacity 128; metadata uses 10 of 4,096 blocks (0.24 %) on a 16 MiB image.

ID	Quality	How it applies to SchedFS	Measurable criterion
NFR6	Maintainability	Code is split into layered modules with small interfaces and a fixed style.	8 headers, one module per directory, `clang-format` style; builds with `-Wall -Wextra -Wpedantic -Wshadow -Wconversion` and no warnings.
NFR7	Usability	Commands follow normal Linux conventions and give clear error messages.	Usage text on invalid options; standard `errno` messages (Fig. 34, Fig. 22).
NFR8	Portability	Runs on any Linux system with libfuse 3 and a C11 compiler.	Built and tested on Ubuntu 24.04 with GCC 13.3 and libfuse 3.14.

3(c) Description of the Modules of the Project

SchedFS is organised into ten modules. Nine follow directly from the source tree. The inode and directory layer (M3) and the block-allocation and mapping layer (M4) share `src/fs/filesystem.c`, but they are separate groups of functions with separate locks, so they are described separately. Table 4 gives an overview, and the paragraphs that follow describe each module.

Table 4 Module overview

ID	Module	Main source files	LOC
M1	Image formatter	`mkfs/mkfs.c`, `include/layout.h`	180
M2	FUSE adapter	`src/fuse/fuse_main.c`	199
M3	Inode and directory layer	`src/fs/filesystem.c` (paths, inodes, dirs)	808
M4	Block allocation and mapping	`src/fs/filesystem.c` (bitmap, `bmap`)	
M5	Request lifecycle and bounded queue	`src/sync/request_queue.c`	149
M6	Disk scheduler	`src/sched/scheduler.c`	113
M7	Virtual disk and dispatcher	`src/storage/virtual_disk.c`	124
M8	Metrics and logging	`src/bench/metrics.c`, `src/utils/logger.c`	84
M9	Benchmark and visualisation	`src/bench/benchmark_main.c`, `scripts/`	246
M10	Test suite and build	`tests/`, `Makefile`	200
	Total		**2,103**

Counting note. LOC are non-blank, non-comment lines, including each module's header files (Section 6.4). M3 and M4 are counted together because they share one source file and `include/filesystem.h`.

Traceability to the synopsis. The approved synopsis planned ten modules and 26 functional requirements (FR-01 to FR-26). During implementation some planned modules were merged into fewer source files. For example, all four policies live in `scheduler.c` rather than one file per policy, and the superblock, inode and directory code share `filesystem.c`. Some functions also received different names: the synopsis's `pick_next()` became `scheduler_pick()`, and `blk_alloc` became `alloc_block`. Table 5 maps each planned module to its implementation, and Table 6 maps the synopsis requirements to the requirements of this report.

Table 5 Synopsis modules and where they are implemented

No.	Synopsis module	Report module	Implemented in (main functions)
1	FUSE interface	M2	`fuse_main.c`: `op_getattr`, `op_read`, `op_write`, ...
2	Superblock	M1, M3	`mkfs.c`: `main`; `filesystem.c`: `sfs_mount_image`, `sync_sb`
3	Inodes	M3, M4	`filesystem.c`: `inode_alloc`, `inode_read`, `inode_write`, `bmap`
4	Directories	M3	`filesystem.c`: `resolve`, `dir_find`, `dir_add`, `dir_remove`
5	Virtual disk	M7	`virtual_disk.c`: `vdisk_open`, `vdisk_read`, `vdisk_write`
6	Block allocation	M4	`filesystem.c`: `alloc_block`, `free_block`, `free_inode_blocks`
7	Request queue	M5	`request_queue.c`: `rq_submit`, `rq_take`, `request_wait`
8	Disk scheduling	M6	`scheduler.c`: `scheduler_pick`
9	Synchronization	M5, M7 (locks in M3, M4)	dispatcher thread in `virtual_disk.c`; mutexes and rwlocks in `filesystem.c`
10	Benchmarking	M8, M9	`benchmark_main.c`, `metrics.c`, `scripts/`

Table 6 Synopsis functional requirements and the corresponding requirements in this report

Synopsis	**This report**	**Note**
FR-01 Mounting	FR2	
FR-02 Initialisation	FR1	formatting by `mkfs.schedfs`
FR-03, FR-05, FR-06 Create, read, write	FR3	
FR-04 File opening	FR3	existence and file type are checked; access-mode bits are not enforced
FR-07 File deletion	FR5	
FR-08 to FR-10 Directories	FR4	
FR-11 Path resolution	FR3, FR4	`resolve()`
FR-12 Inode management	FR3, FR6	
FR-13, FR-14 Block allocation and release	FR3, FR5	module M4
FR-15 Virtual disk I/O	FR1, FR9	
FR-16, FR-17, FR-24 Requests, queue, producer–consumer	FR7	
FR-18 to FR-21 FCFS, SSTF, SCAN, Adaptive	FR8	
FR-22, FR-23 Concurrent I/O, synchronization	FR7, NFR2, NFR3	
FR-25 Logging	FR10	mount summaries, allocations (verbose) and errors are logged; individual scheduling decisions are not
FR-26 Metrics	FR10, FR11	

M1. Image Formatter (`mkfs.schedfs`)

Purpose	Create a new, empty SchedFS image with a valid layout.
Inputs	Image path and size, for example `disk.img 16M` (suffixes M, MiB, G, GiB).
Outputs	An image file containing the superblock, bitmap, inode table and root directory, plus a one-line summary on standard output.
Internal working	Parses and validates the size, computes the number of bitmap and inode-table blocks, opens the file with `O_CREAT|O_EXCL` so an existing image is never overwritten, extends it with `ftruncate`, writes block 0 (superblock), the bitmap with all metadata blocks marked used, the root inode (inode 1) and the root directory entries `.` and `..`, then calls `fsync`.
OS concepts	File-system layout, free-space bitmap, indexed allocation, the `open`, `pwrite` and `fsync` system calls.
Interactions	Produces the image consumed by M3–M7; shares the on-disk structures in `layout.h`.
Source files	`mkfs/mkfs.c` (`parse_size`, `write_full`, `main`); `include/layout.h`.

M2. FUSE Adapter

Purpose	Connect the Linux VFS to the SchedFS API and manage the mount lifecycle.
Inputs	Command-line options (image, mount point, policy, `alpha`, `beta`, `--verbose`, FUSE options) and FUSE callbacks from the kernel.
Outputs	Return codes and data for each VFS operation; a mounted file system of type `fuse.schedfs`.
Internal working	`main()` parses the options, resolves the image to an absolute path, validates it once so errors reach the terminal, and calls `fuse_main()`. After FUSE daemonizes, the `init` callback mounts the image and starts the dispatcher; `destroy` unmounts it. Each callback such as `op_read` or `op_mkdir` forwards to the matching `sfs_*` function.
OS concepts	User-space file systems, system-call dispatch through the VFS, `fork` during daemonization, process working directory.
Interactions	Calls M3 for every operation; relies on M7 being started inside `op_init`.
Source files	`src/fuse/fuse_main.c` (`op_init`, `op_destroy`, `op_getattr`, ..., `main`).

M3. Inode and Directory Layer

Purpose	Implement files, directories and path names on top of numbered blocks.
Inputs	Absolute paths, byte offsets and buffers from M2.
Outputs	File data, `struct stat`, directory listings and `errno` codes such as `ENOENT`, `EEXIST`, `ENOTEMPTY`, `ENAMETOOLONG`.
Internal working	`resolve()` walks the path from the root inode, one component at a time, using `dir_find()`. Directories are arrays of 64-byte entries. `inode_alloc()` finds a free slot among 256 inodes; `inode_read`/`inode_write` copy a 128-byte inode in or out of its 4 KiB table block. Read, write and truncate take a reader–writer lock per inode.
OS concepts	Inodes, directories, path-name resolution, reader–writer locks, metadata consistency.
Interactions	Calls M4 to map logical blocks and M7 (`vdisk_read`/`vdisk_write`) for every block.
Source files	`src/fs/filesystem.c` (`resolve`, `dir_find`, `dir_add`, `dir_remove`, `inode_*`, `sfs_*`); `include/filesystem.h`.

M4. Block Allocation and Mapping

Purpose	Manage free space and translate a file's logical block numbers into disk blocks.
Inputs	An inode, a logical block number and a *create* flag.
Outputs	A physical block number (0 for a hole), or `ENOSPC`/`EFBIG`.
Internal working	`bmap()` uses `direct[0..11]` for the first 12 blocks and the single-indirect block (1,024 pointers) after that. `alloc_block()` searches the in-memory bitmap from a *hint*—the previous block of the same file—and wraps around once, so consecutive file blocks tend to be adjacent. Each allocation updates the bitmap and superblock on disk and zero-fills the new block.
OS concepts	Indexed allocation, free-space management with a bitmap, locality-aware allocation.
Interactions	Used by M3; writes through M7.
Source files	`src/fs/filesystem.c` (`bmap`, `alloc_block`, `free_block`, `free_inode_blocks`, `sync_bitmap`, `sync_sb`).

M5. Request Lifecycle and Bounded Queue

Purpose	Carry one block I/O from a file-system thread to the dispatcher and back.
Inputs	A request (operation, block number, buffer) from M7's `transfer()`.
Outputs	The request handed to the dispatcher; the caller woken with the result.
Internal working	`request_init` stamps an ID and a monotonic arrival time. `rq_submit` appends to an array of at most 128 entries, waiting on `not_full` when the array is full. `rq_take` waits on `not_empty`, asks M6 which request to serve, removes it and signals `not_full`. `request_wait` sleeps on the request's own condition variable until `completed` is set.
OS concepts	Bounded-buffer (producer–consumer) problem, mutexes, condition variables, predicate re-checking after wake-up.
Interactions	Called by M7; calls M6 under the queue mutex.
Source files	`src/sync/request_queue.c`; `include/request.h`, `include/request_queue.h`.

M6. Disk Scheduler

Purpose	Choose which pending request to serve next.
Inputs	The pending requests, the current head position and direction, and the current time.
Outputs	The index of the chosen request.
Internal working	FCFS chooses the oldest request. SSTF chooses the smallest $\lvert\text{head} - \text{block}\rvert$. SCAN chooses the nearest request in the current direction and, if none remains, the nearest request overall (the dispatcher then reverses direction). Adaptive minimises $\alpha \cdot d/N - \beta \cdot w/w_{\max}$. Every tie is broken by arrival time and then request ID.
OS concepts	Disk-scheduling algorithms, starvation versus throughput, ageing.
Interactions	Called by M5 (live) and by M9 and the unit test (replay).
Source files	`src/sched/scheduler.c` (`scheduler_pick`, `scheduler_parse`, `older`, `distance`); `include/scheduler.h`.

M7. Virtual Disk and Dispatcher

Purpose	Own the image file and perform all block I/O on one dedicated thread.
Inputs	Block reads and writes from M3/M4 (`vdisk_read`, `vdisk_write`).
Outputs	4 KiB block transfers to and from `disk.img`; completion signals; metric samples.
Internal working	`vdisk_open` opens the image and creates the dispatcher thread. `transfer()` builds a request on the caller's stack, submits it and waits. The dispatcher loops on `rq_take`, performs a full `pread`/`pwrite` (retrying on `EINTR` and short transfers), records the seek distance and waiting time, updates the head position and SCAN direction, and signals the caller. `vdisk_close` stops the queue, joins the thread and calls `fsync`.
OS concepts	Threads, single-consumer service loop, the `pread`/`pwrite` system calls, graceful shutdown.
Interactions	Consumes M5, calls M6 through M5, reports to M8.
Source files	`src/storage/virtual_disk.c`; `include/virtual_disk.h`.

M8. Metrics and Logging

Purpose	Measure scheduling behaviour and record events.
Inputs	Seek samples (from block, to block, waiting time) and log messages.
Outputs	Running totals and `filesystem.log` entries with timestamps and levels.
Internal working	`metrics_record` updates request count, total seek, waiting-time sum, sum of squares and maximum under a mutex. The logger formats ISO-8601 timestamps and writes INFO, WARNING and ERROR entries, plus DEBUG entries when `--verbose` is given.
OS concepts	Performance measurement with `CLOCK_MONOTONIC`, thread-safe shared counters.
Interactions	Fed by M7 and M3; read at unmount to log the summary line.
Source files	`src/bench/metrics.c`, `src/utils/logger.c`; `include/metrics.h`, `include/logger.h`.

M9. Benchmark and Visualisation

Purpose	Compare the policies on identical traces and present the results.
Inputs	Five fixed block traces (sequential, random, clustered, mixed, concurrent).
Outputs	`benchmark.csv`; PNG graphs; an interactive web page.
Internal working	`schedfs-bench` replays each trace under each policy with a starting head of 53, calls `scheduler_pick` repeatedly, adds SCAN's travel to the disk edge, applies a seek-delay model of 10 µs plus 2 µs per block, and writes ten metrics per run. `plot_results.py` reads the CSV with pandas and draws bar charts with Matplotlib. `docs/index.html` animates the same selection logic in the browser.
OS concepts	Disk-scheduling evaluation: seek distance, waiting time, throughput, fairness.
Interactions	Reuses M6 directly, without the file system.
Source files	`src/bench/benchmark_main.c`, `scripts/plot_results.py`, `scripts/generate_workload.py`, `docs/index.html`.

M10. Test Suite and Build

Purpose	Build all binaries and verify behaviour at three levels.
Inputs	Source files; a freshly formatted image; `/dev/fuse`.
Outputs	Executables and pass/fail results.
Internal working	The Makefile builds `schedfs`, `mkfs.schedfs` and `schedfs-bench` with strict warnings. `test_scheduler` checks the textbook trace; `test_core` runs persistence and 4-thread concurrency checks against the core API; `system.sh` mounts a real image under each policy, and once in background mode, and runs shell commands against it.
OS concepts	Unit, integration and system testing of concurrent code.
Interactions	Exercises M1–M8.
Source files	`Makefile`, `tests/test_scheduler.c`, `tests/test_core.c`, `tests/system.sh`.

4. TOOLS AND TECHNOLOGIES USED

Every technology in Table 7 was confirmed in the source tree, the Makefile or the build log in Fig. 19.

Table 7 Tools and technologies

Technology	Purpose	Usage in the project
C11 (GNU dialect, `-std=gnu11`)	Implementation language	All of `src/`, `mkfs/` and the C tests; `_Static_assert` checks the on-disk structure sizes; `stdatomic.h` provides the request-ID counter [6].
GCC 13.3	Compiler	Builds with `-O2 -g -Wall -Wextra -Wpedantic -Wshadow -Wconversion`; no warnings are produced (Fig. 19).
GNU Make	Build automation	Targets `all`, `test`, `bench`, `format`, `install`, `clean`.
libfuse 3 (3.14.0)	User-space file-system API	`fuse_main()` and a `struct fuse_operations` table with 16 callbacks in `fuse_main.c`; found through `pkg-config fuse3` [7, 8].
Linux FUSE module, `fusermount3`	Kernel bridge and mount helper	Routes VFS calls on the mount point to the SchedFS process; `fusermount3 -u` unmounts [9].
POSIX threads	Concurrency and synchronization	`pthread_create/join` for the dispatcher; mutexes and condition variables for the queue and request completion; `pthread_rwlock_t` per inode [10].
POSIX/Linux system calls	Storage access	`open`, `pread`, `pwrite`, `ftruncate`, `fsync`, `close`, `realpath`; `clock_gettime(CLOCK_MONOTONIC)` for timing [4].
Bash and GNU coreutils	System testing	`tests/system.sh` uses `mountpoint`, `cmp`, `truncate`, `stat`, `timeout`.
Python 3, pandas, Matplotlib	Analysis and graphs	`scripts/plot_results.py` reads `benchmark.csv` and writes the PNG charts in `results/` [11, 12].
HTML, CSS, JavaScript	Interactive visualisation	`docs/index.html`: plain JavaScript (no framework) that ports `scheduler_pick()`; published with GitHub Pages.
ThreadSanitizer (`-fsanitize=thread`)	Data-race detection	Used in this report to check the 4-thread core test (Fig. 35) [13].
clang-format	Code style	`.clang-format`: LLVM base style, 4-space indent, 100-column limit.
Git and GitHub	Version control and hosting	Feature branches merged through pull requests (Fig. 29); GitHub Pages hosts the demo.

Technology	Purpose	Usage in the project
Docker (`ubuntu:24.04`)	Reproducible Linux test bed	All runs in this report were made in a container with `/dev/fuse`, as an unprivileged user.
LaTeX (pdfLaTeX, TikZ, PGFPlots)	Documentation	This report; all diagrams and charts are drawn from the project's data.

Why these choices. C with POSIX threads gives direct control over memory layout, which matters for an on-disk format with fixed 128-byte inodes, and it uses the same synchronization primitives taught in the course. FUSE lets a file system run as an ordinary process, so it can be debugged and restarted without kernel programming and without risk to the host, at the cost of extra context switches per operation [14]. A single image file makes the disk a plain array of numbered blocks, so "seek distance" has a precise meaning. Python and a browser page are used only for analysis and presentation; they never touch the file system's data path.

5. DESIGN OF THE PROJECT

5.1 System Architecture

SchedFS is a strictly layered system (Fig. 1). A system call on the mount point travels through the Linux VFS [15, 16] and the FUSE kernel module to a libfuse worker thread inside the SchedFS process. The FUSE adapter forwards it to the inode and directory layer, which breaks it into 4 KiB block operations. Each block operation becomes a request in the bounded queue. Only after the scheduler has chosen it does the dispatcher thread perform the real `pread` or `pwrite` on `disk.img`. After the mount-time bootstrap read of the superblock, no other code touches the image, so the scheduler controls every byte of disk I/O.

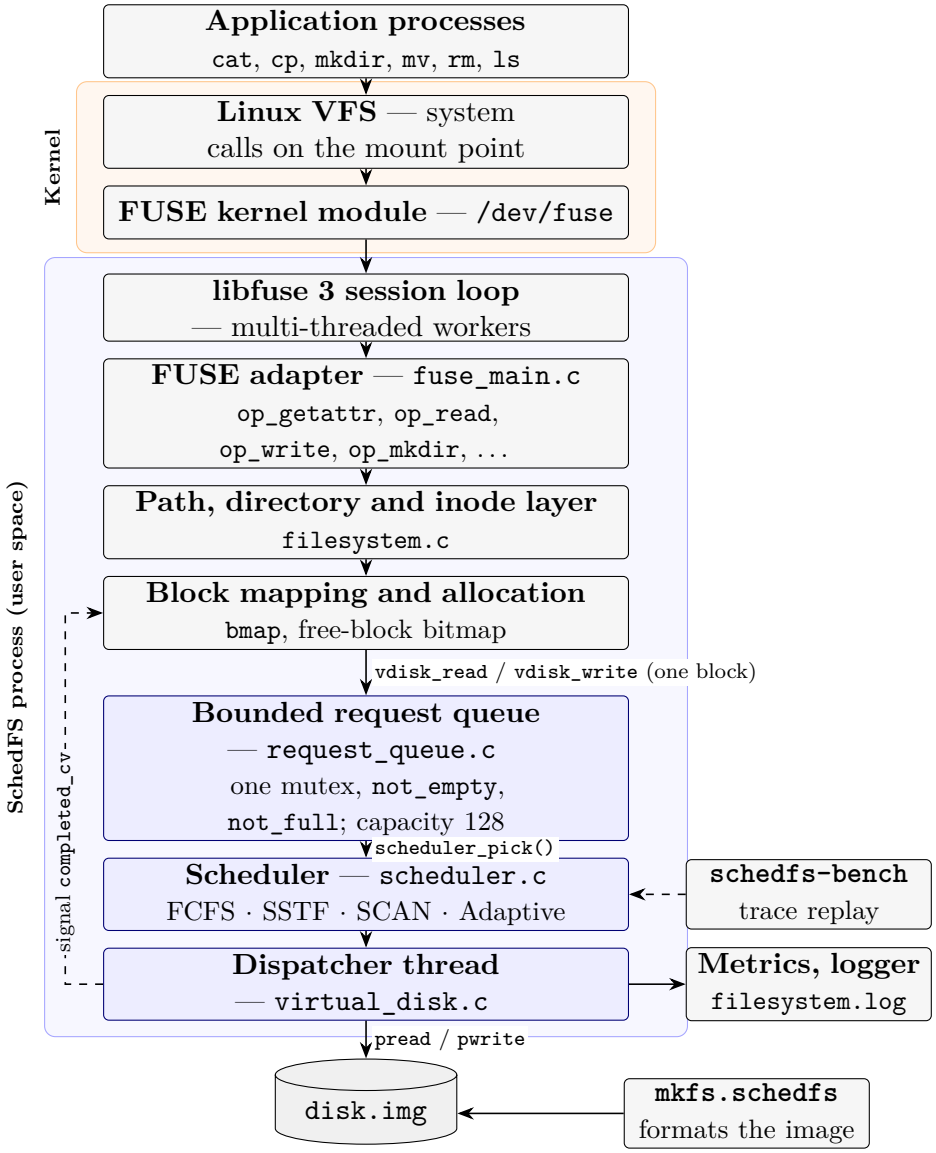


Fig. 1 System architecture of SchedFS. Shaded blue layers are where scheduling takes place.

Fig. 2 shows the component architecture as a dependency graph of the source modules. Arrows point from a module to the module whose interface it calls. The graph has no cycles: the scheduler depends only on the request definition, which is why the benchmark and the unit test can reuse it without a file system.

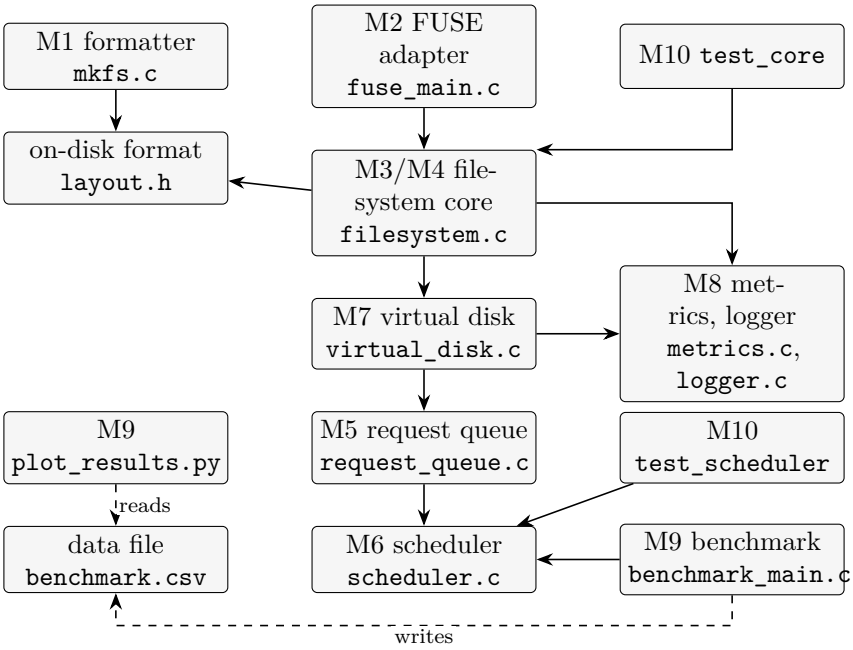


Fig. 2 Component architecture: dependencies between source modules (dashed: data files).

5.2 System Workflow

The life of a SchedFS instance has four phases: build, format, mount and serve, and unmount (Fig. 3). Validation happens before FUSE takes over, so that every configuration error is printed in the terminal rather than lost inside a background process.

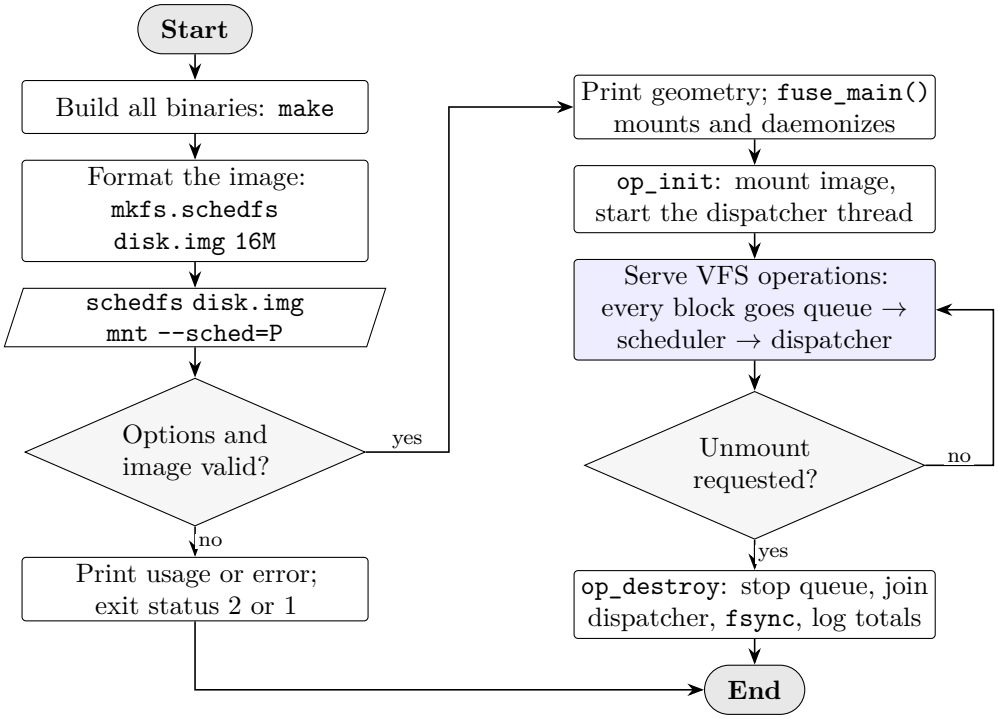


Fig. 3 Overall workflow from build to unmount.

Fig. 4 shows a detail of the mount phase that shaped the design. Without `-f`, `fuse_main()` daemonizes with `fork()`, and only the calling thread survives a `fork()`. If the dispatcher thread were created before `fuse_main()`, the daemon would have a request queue but nobody to empty it, and the first `mkdir` would block forever. SchedFS therefore validates the image in the foreground, releases it, and starts the real instance—including the dispatcher—in the FUSE `init` callback, which runs inside the daemon. The image path is converted to an absolute path first, because the daemon changes its working directory to `/`.

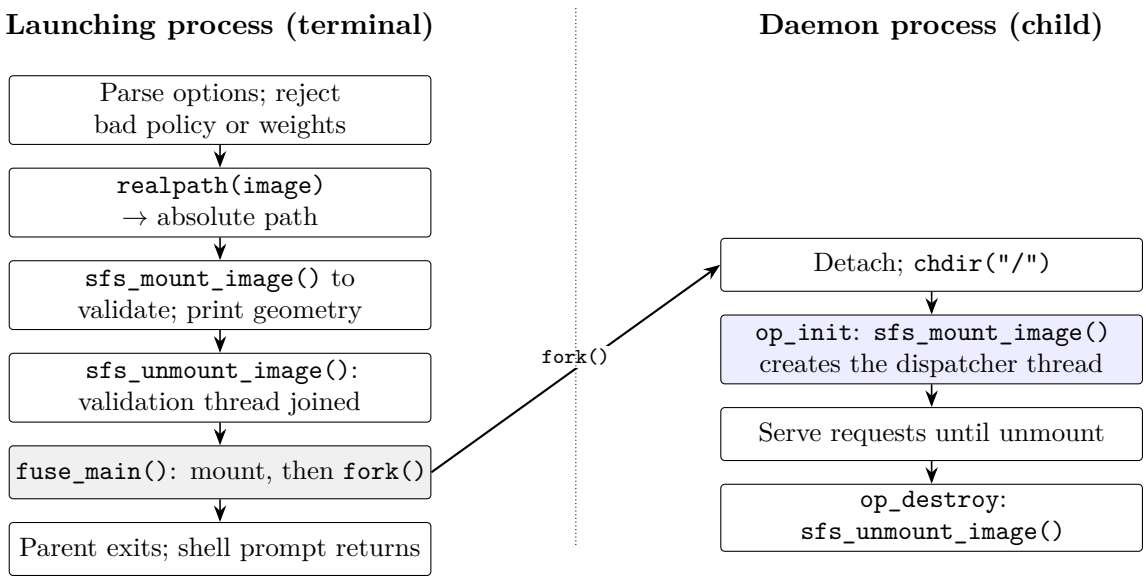


Fig. 4 Mount sequence. The dispatcher thread is created in `op_init`, after the fork.

5.3 Module-Wise Design

5.3.1 On-disk format (M1, M3, M4)

A 16 MiB image holds 4,096 blocks of 4,096 bytes (Fig. 5). Block 0 is the superblock, block 1 is the free-block bitmap (one bit per block, so one bitmap block covers 32,768 blocks), blocks 2–9 hold 256 inodes of 128 bytes each (32 per block), and blocks 10–4095 are data blocks. Block 10 is the root directory. Table 8 decodes the first 64 bytes of a freshly formatted image, read with `od` in Fig. 20.

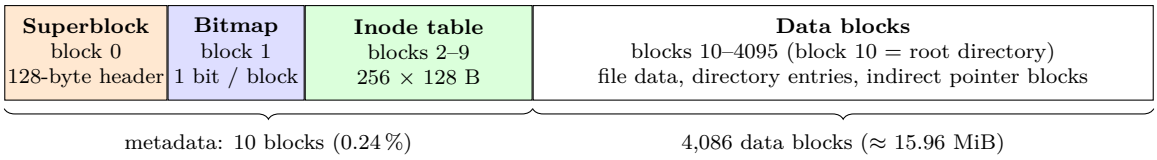


Fig. 5 On-disk layout of a 16 MiB SchedFS image (widths not to scale).

Table 8 Superblock of a freshly formatted 16 MiB image (values from `od`, Fig. 20)

Offset	Field	Value	Meaning
0	`magic`	`0x53465331`	ASCII “SFS1” identifies a SchedFS image
4	`version`	1	format version
8	`block_size`	`0x1000` = 4096	bytes per block
12	`total_blocks`	`0x1000` = 4096	16 MiB / 4 KiB
16	`free_blocks`	`0xff5` = 4085	data blocks minus the root directory block
20	`total_inodes`	`0x100` = 256	fixed inode count
24	`free_inodes`	`0xff` = 255	all but the root inode
28, 32	`bitmap_start`, `bitmap_blocks`	1, 1	bitmap location
36, 40	`inode_start`, `inode_blocks`	2, 8	inode-table location
44	`data_start`	`0xa` = 10	first data block
48	`root_inode`	1	inode number of `/`

Each inode uses *indexed allocation* with twelve direct pointers and one single-indirect pointer (Fig. 6), following the classic UNIX inode design [17]. The indirect block holds 1,024 four-byte block numbers, so a file can have $12 + 1024 = 1036$ blocks, or about 4.05 MiB. A pointer of zero marks a hole: reading it returns zeros without allocating a block, which is how a sparse `truncate` works.

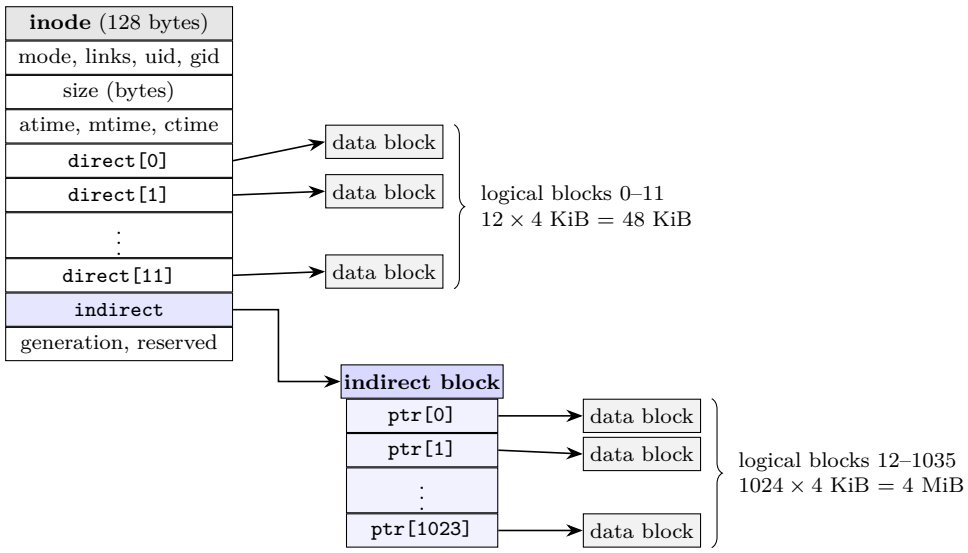


Fig. 6 Inode structure and indexed allocation (12 direct pointers + 1 single-indirect block).

5.3.2 Image formatter (M1)

The formatter performs every check before it writes anything (Fig. 7). Opening with `O_EXCL` makes it impossible to destroy an existing image by mistake, and any failure after creation removes the partly written file.

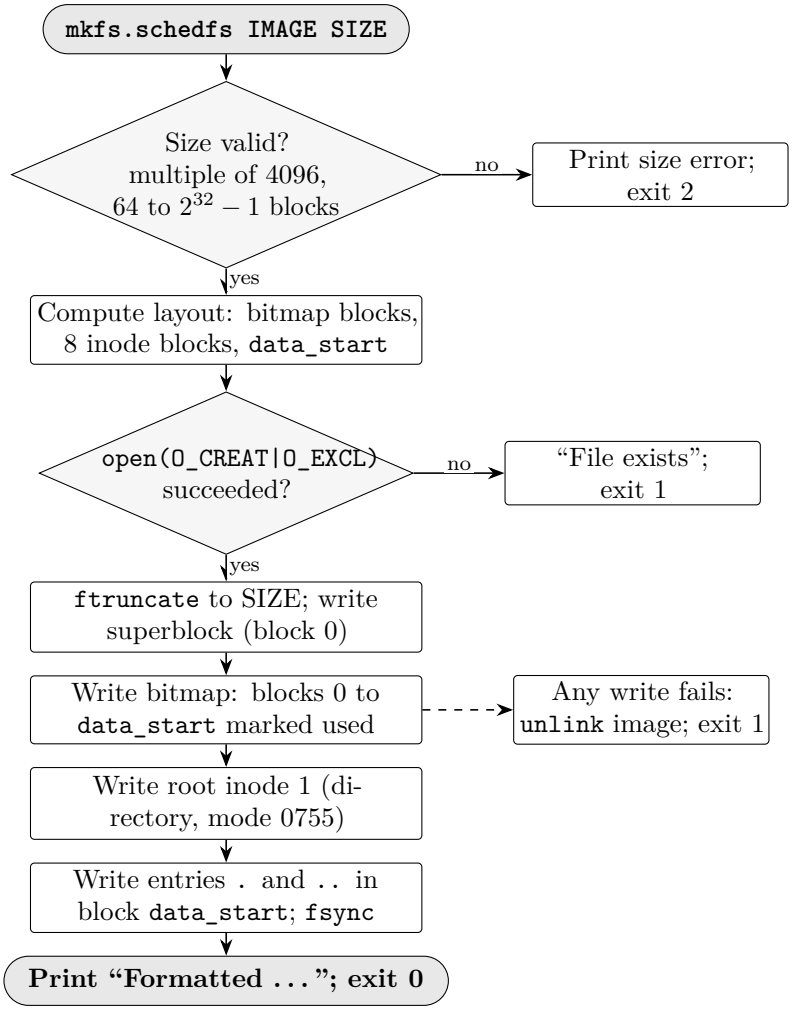


Fig. 7 Flowchart of the image formatter (`mkfs/mkfs.c`).

5.3.3 Path resolution and the write path (M3, M4)

Paths are resolved from the root, one component at a time (Fig. 8). A write (Fig. 9) holds the file's inode lock in exclusive mode, maps each logical block with `bmap(create = true)`, and reads the old block first only when the write covers part of it (read–modify–write). Size and modification time are written back once, at the end.

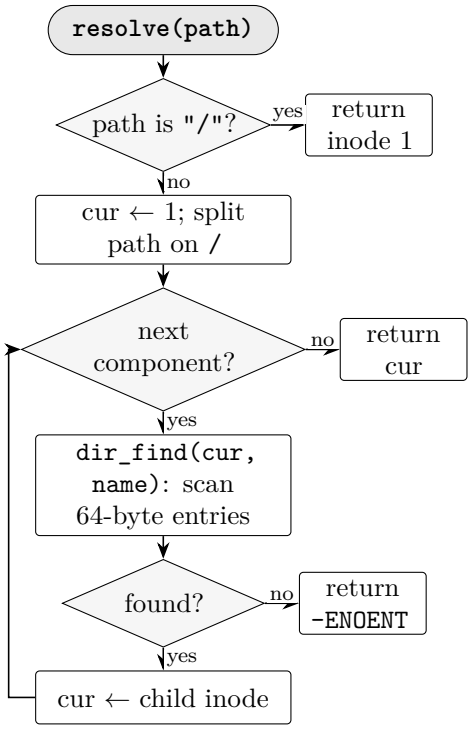


Fig. 8 Path resolution.

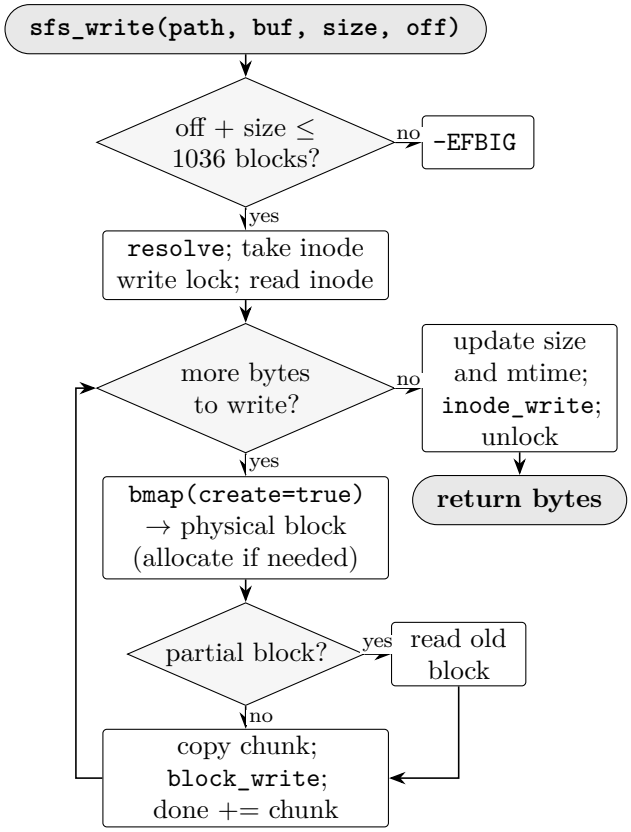


Fig. 9 Write path through M3 and M4.

5.3.4 Block allocator (M4)

`alloc_block()` looks for a free bit starting from a *hint*: the physical block that holds the file's previous logical block (Fig. 10). Consecutive blocks of a file are therefore usually adjacent on disk. This keeps seek distances small for sequential access and makes the effect of scheduling easier to see. The Berkeley Fast File System applies the same locality principle [18]. The bitmap and superblock are written to disk before the block is used, and a failed write rolls the in-memory state back.

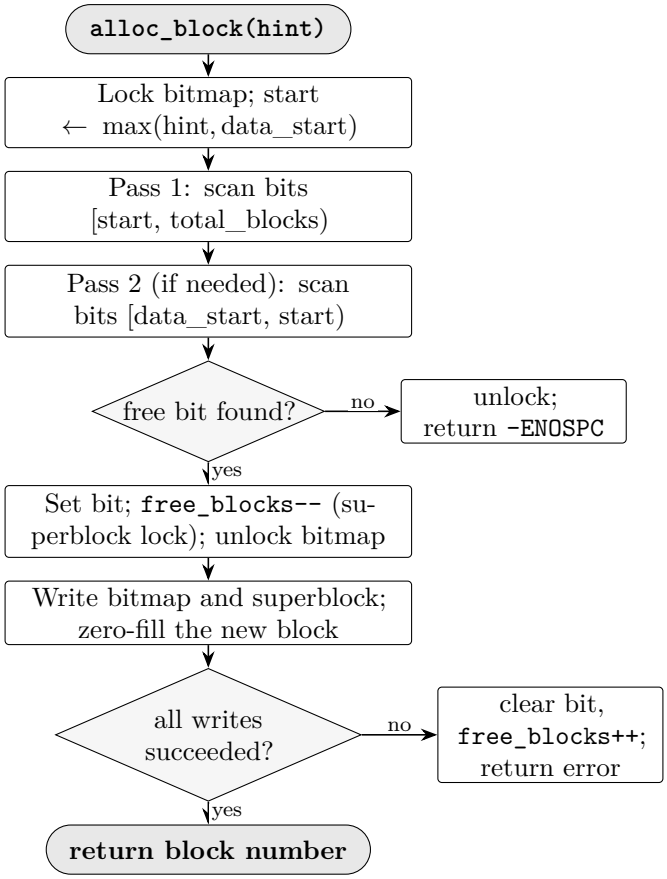


Fig. 10 Hint-based, two-pass block allocation.

5.3.5 Bounded request queue (M5)

The queue is the classic bounded buffer: producers wait while it is full, the consumer waits while it is empty, and every wait sits inside a `while` loop that re-checks its condition (Fig. 11). A `stopping` flag lets unmount wake every waiter. Producers then receive `-ESHUTDOWN`, and the dispatcher receives `NULL` once the queue has drained.

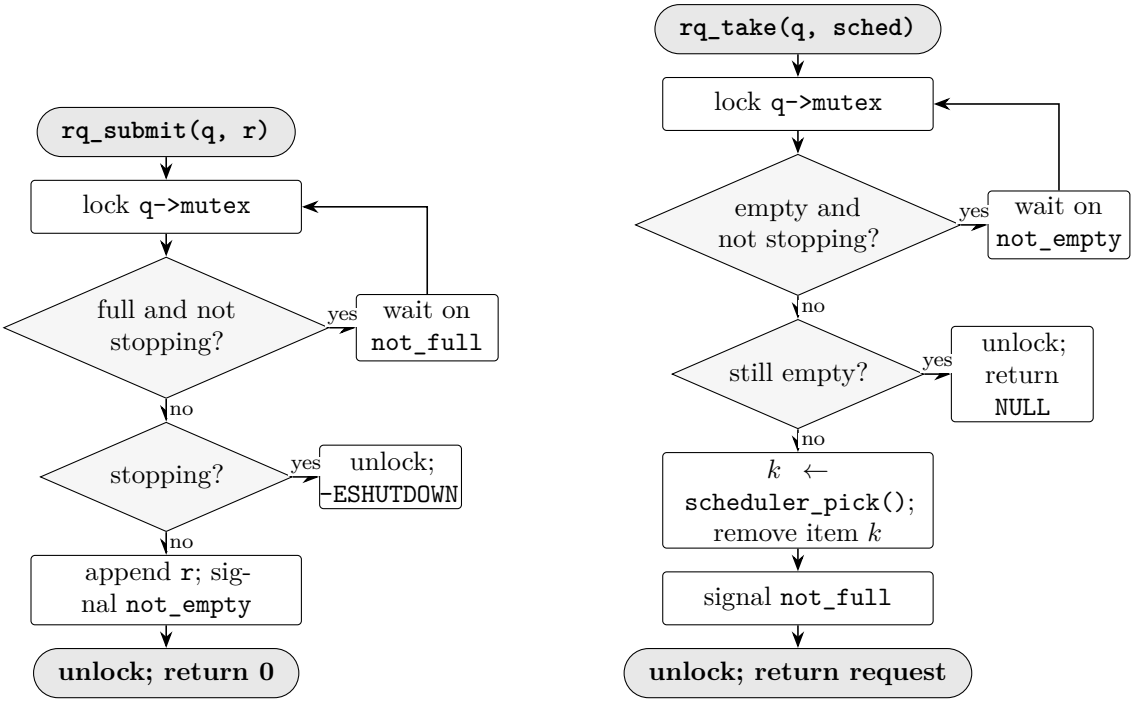


Fig. 11 Producer side (left) and consumer side (right) of the bounded request queue.

5.3.6 Disk scheduler (M6)

`scheduler_pick()` is a pure function: it reads the pending requests, the head position, the SCAN direction and the current time, and returns an index (Fig. 12). It neither sleeps nor performs I/O, so it can run under the queue mutex and can be tested on its own. The four policies differ only in how they rank candidates; all of them break ties by arrival time and then request ID, which makes the service order deterministic.

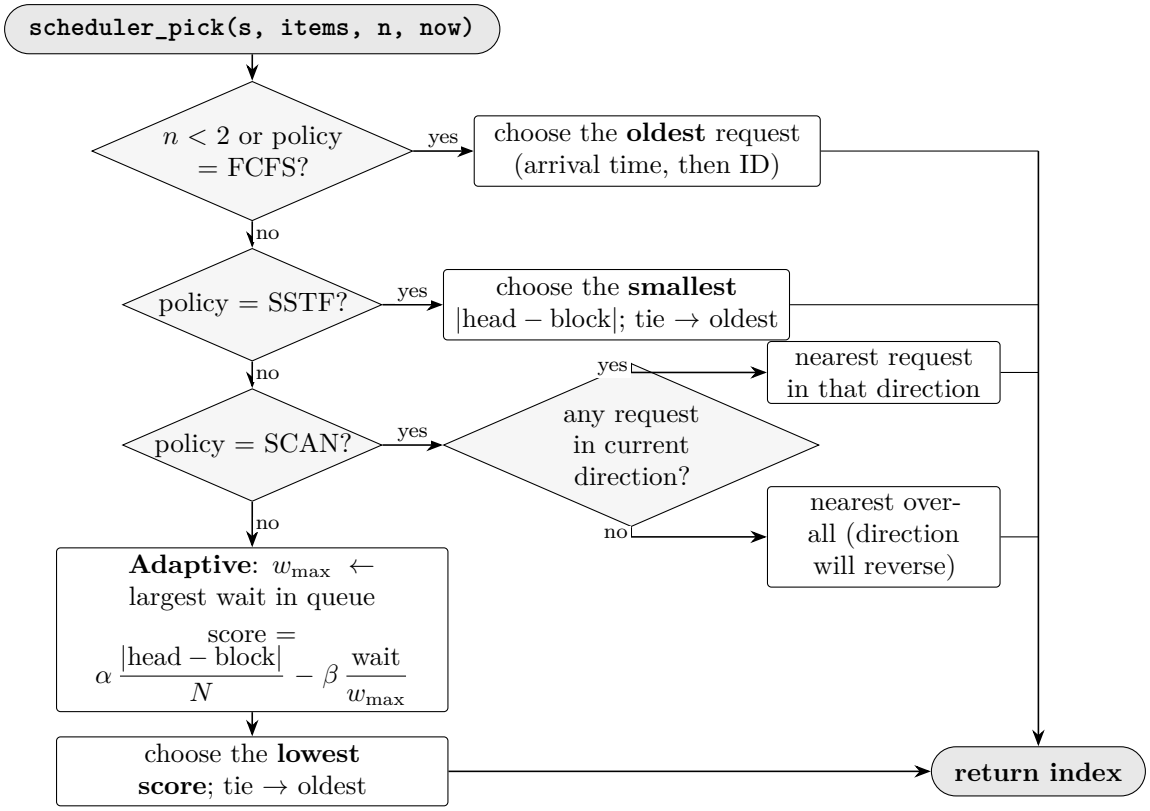


Fig. 12 Decision logic of `scheduler_pick()` for the four policies.

5.3.7 Dispatcher (M7) and benchmark replay (M9)

The dispatcher is the only consumer (Fig. 13, left). After each transfer it records the seek distance and waiting time, moves the head to the served block, reverses the SCAN direction when the head has turned back, and wakes the waiting caller. The benchmark (Fig. 13, right) reuses the same `scheduler_pick()` on fixed traces. All requests arrive at the start, so every policy sees identical input.

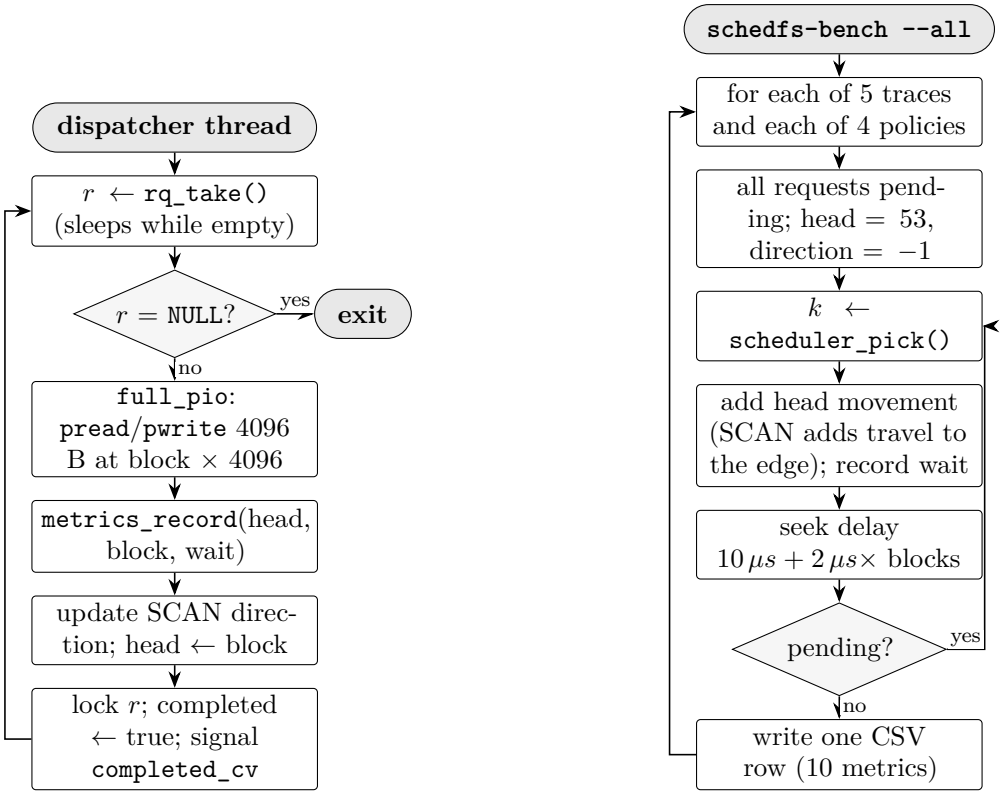


Fig. 13 Dispatcher service loop (left) and benchmark replay loop (right).

5.4 Module Interaction

Fig. 14 follows one block read or write through the modules. The caller's request lives on the caller's own stack, and the dispatcher writes the data straight into the caller's buffer. No data is copied through the queue; only a pointer is passed. Because the caller sleeps on a condition variable that belongs to its own request, many threads can be waiting at once, and the scheduler always has a choice to make.

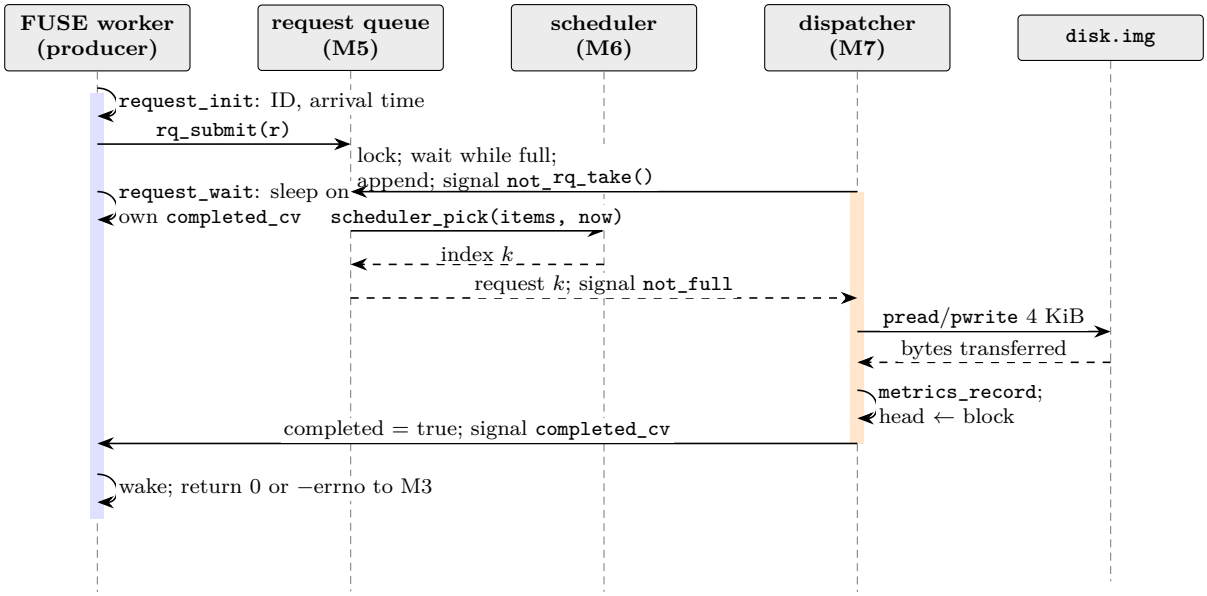


Fig. 14 Sequence diagram of one scheduled block I/O.

Table 9 lists the interfaces through which the modules exchange information.

Table 9 Interfaces between modules

From → To	Interface	Information exchanged
Kernel → M2	`struct fuse_operations`	path, offset, size, buffer, mode, flags
M2 → M3	`sfs_getattr`, `sfs_read`, `sfs_write`, ...	same arguments; returns bytes or `-errno`
M3 → M4	`bmap`, `alloc_block`, `free_block`	inode, logical block, create flag → physical block
M3/M4 → M7	`vdisk_read`, `vdisk_write`	block number, 4 KiB buffer pointer
M7 → M5	`rq_submit`, `request_wait`, `rq_take`	pointer to `disk_request_t`
M5 → M6	`scheduler_pick`	pending array, head, direction, time → index
M7 → M8	`metrics_record`, `sfs_log`	from-block, to-block, wait (ms); log text
M9 → M6	`scheduler_pick`	fixed traces → service order
M9 → files	`benchmark.csv` → `plot_results.py`	10 metrics per (workload, policy)

5.5 Operating Systems Concepts

5.5.1 Disk scheduling

Each policy answers one question: *which pending request should the head serve next?* [1, 19, 20] Table 10 summarises the four answers. Fig. 15 plots the head position over time for the textbook trace (head at block 53, queue 98, 183, 37, 122, 14, 124, 65, 67 on a 200-block disk), using the exact service orders checked by `tests/test_scheduler.c`.

Table 10 Scheduling policies implemented in `scheduler.c`

Policy	Chooses next	Strength	Weakness
FCFS	the oldest request	fair; no starvation	ignores head position, so seeks are long
SSTF	the request nearest the head	short seeks	far requests can starve
SCAN	the nearest request in the current direction; reverses at the edge	sweeps evenly; bounded wait	extra travel to the edge
Adaptive	lowest $\alpha \cdot d/N - \beta \cdot w/w_{\max}$	tunable between SSTF ($\beta = 0$) and age priority	sensitive to the scale of d and w

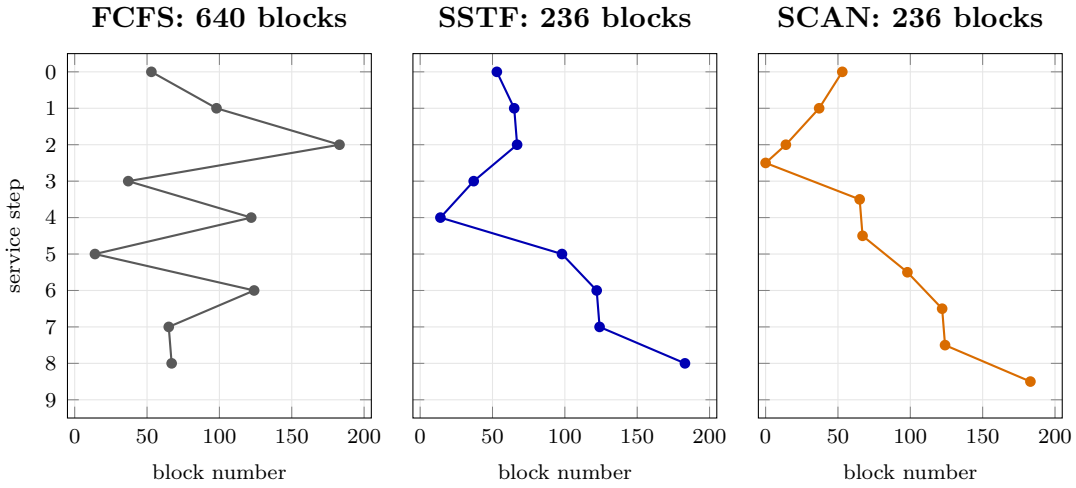


Fig. 15 Head movement for the textbook trace. SCAN first moves towards block 0 and reaches the edge before reversing.

The totals follow directly from the orders. For FCFS: $|98-53|+|183-98|+|37-183|+|122-37|+|14-122|+|124-14|+|65-124|+|67-65| = 45+85+146+85+108+110+59+2 = 640$. For SSTF: $12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236$. For SCAN, moving down first: $53 + 183 = 236$ (down to block 0, then up to block 183).

5.5.2 The Adaptive policy and ageing

SSTF minimises seek distance but can starve a request far from the head. FCFS never starves, but it ignores position. The Adaptive policy combines both quantities in one score:

$$\text{score}(r) \;=\; \alpha \,\frac{|\text{head} - \text{block}(r)|}{N} \;-\; \beta \,\frac{\text{now} - \text{arrival}(r)}{w_{\max}}, \tag{1}$$

where N is the number of blocks on the disk, $w_{\max}$ is the longest current wait in the queue, and the request with the lowest score is served. Both terms are normalised to $[0, 1]$. With $\beta = 0$ the policy is exactly SSTF. As β grows, waiting time increasingly outweighs

distance—a form of *ageing*, the standard remedy for starvation [1]. The defaults are $\alpha = 0.7$ and $\beta = 0.3$.

5.5.3 Synchronization: bounded buffer, completion and reader–writer locks

SchedFS uses three synchronization patterns from the course [3, 10, 21, 22]:

1. **Bounded buffer (producer–consumer).** Many FUSE worker threads produce requests and one dispatcher consumes them. A mutex protects the array; condition variables `not_full` and `not_empty` block producers and the consumer respectively (Fig. 11).

2. **Completion signalling.** Each request carries its own mutex, condition variable and `completed` flag. The caller waits in `while (!completed) pthread_cond_wait(...)`, so a spurious wake-up is harmless and a signal cannot be lost: the flag is set under the same mutex before the signal is sent.

3. **Reader–writer locks.** Each inode has a `pthread_rwlock_t`. Reads take it in shared mode, and writes and truncation take it in exclusive mode. Several readers of one file can therefore proceed in parallel, while a writer has exclusive access.

Fig. 16 shows the states a request passes through, including the blocking states created by these patterns.

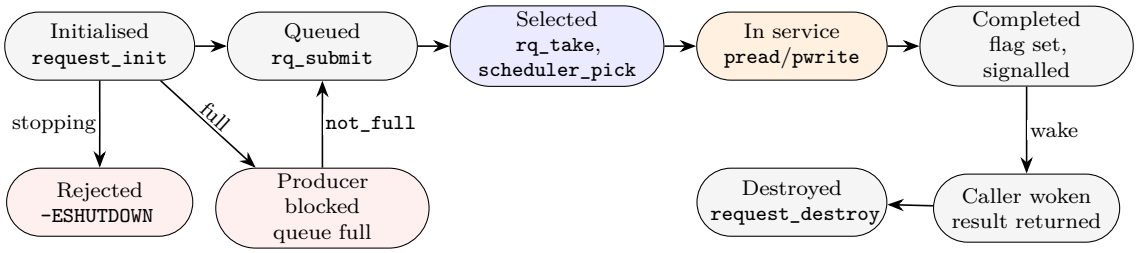


Fig. 16 Life cycle of a disk request.

5.5.4 Deadlock avoidance through lock ordering

A thread that holds one lock and requests another can deadlock if another thread acquires the same two locks in the opposite order. SchedFS prevents this with one global order (Fig. 17), the standard prevention technique of breaking the circular-wait condition [1]. Every code path that holds more than one lock acquires them in that order. A file-system thread may still hold its directory, inode or inode-table lock while it waits for a block request to complete. This cannot deadlock, because the dispatcher that completes the request takes only the queue mutex and the request's own completion mutex, neither of which is held by a waiting thread. The queue mutex is last in the order and is never held during `pread` or `pwrite`: the dispatcher removes the request and releases the mutex

before the transfer.

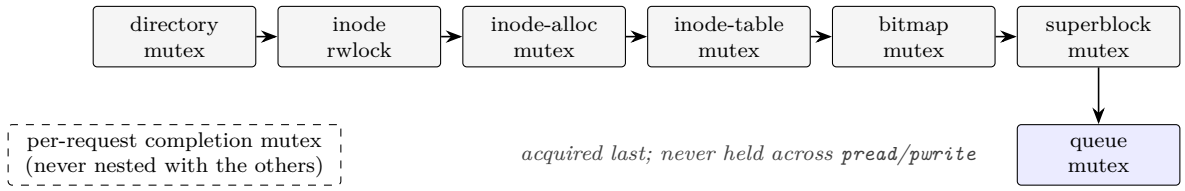


Fig. 17 Global lock-acquisition order used to prevent deadlock.

The inode-table mutex was added after testing exposed a race. Thirty-two inodes share each 4 KiB table block, so updating one inode is a read–modify–write of the whole block. Without the mutex, two threads updating different inodes in the same block could each read the block, change their own inode and write it back, and one update would be lost. `test_core` detected this intermittently, and the mutex removed it.

5.5.5 File systems and system calls

SchedFS implements the textbook file-system abstractions on raw blocks [2,3]: a superblock describing the layout, a free-space bitmap, fixed-size inodes with indexed allocation, directories as files of fixed-size entries, and path-name resolution. Through FUSE, the kernel turns each system call on the mount point (`open`, `read`, `write`, `mkdir`, `rename`, `stat`, `statfs`, ...) into a callback, and SchedFS answers with data or a negative `errno`. The project also illustrates process creation: daemonizing uses `fork()`, and only the calling thread survives it (Fig. 4).

5.6 Data Flow

Fig. 18 is a level-1 data-flow diagram. Circles are processes, open rectangles are data stores, and the plain rectangle is an external entity. Two separate flows exist. The *live path* carries user data to and from the image. The *evaluation path* carries fixed traces through the scheduler into `benchmark.csv` and then into graphs.

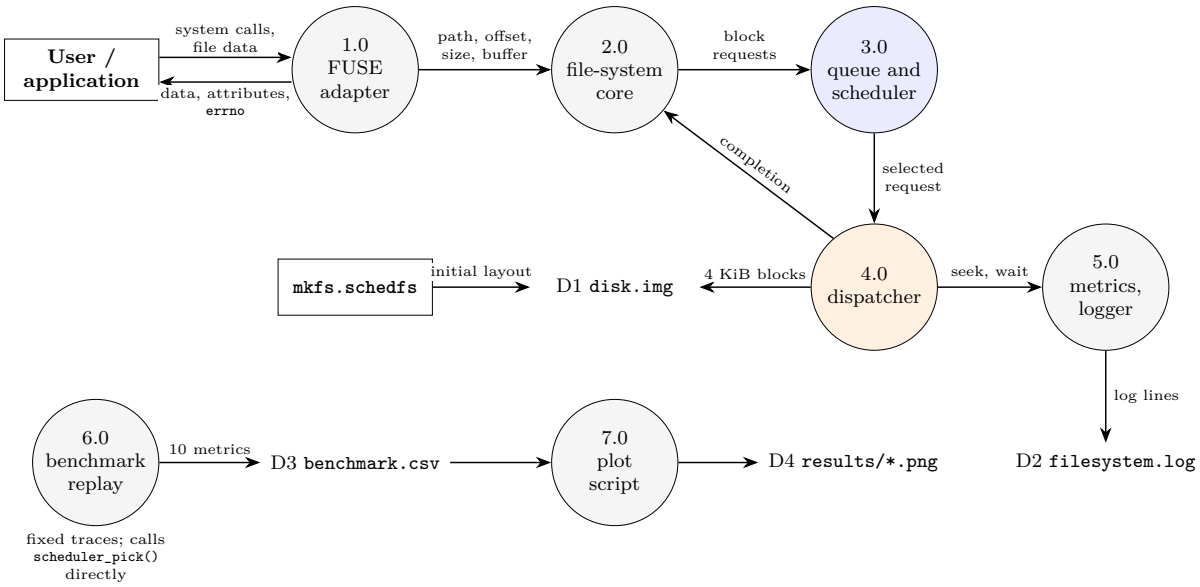


Fig. 18 Level-1 data-flow diagram.

5.7 Design Decisions

Table 11 records the main engineering choices, the alternatives considered and their consequences.

Table 11 Design decisions and trade-offs

Decision	Reason	Trade-off or limitation
Single dispatcher thread	Makes "head position" meaningful and gives the scheduler full control of ordering.	No parallel I/O to the image; throughput is bounded by one thread.
Bounded queue (128 entries)	Back-pressure: producers block instead of consuming unlimited memory.	A burst larger than 128 requests makes producers wait.
Per-request condition variable	Each caller wakes only for its own request; no thundering herd.	One mutex and one condition variable per request.
Request on the caller's stack	No allocation per I/O; the buffer is written in place.	The caller must not return before completion, which `request_wait` guarantees.
Pure `scheduler_pick()`	Testable alone; reused by the benchmark and ported to the web demo.	The dispatcher, not the scheduler, must update head and direction.
Deterministic tie-breaking	Reproducible service orders and exact unit-test expectations.	Ties always favour older requests, a small bias.
Hint-based allocation	Keeps a file's blocks close together, so sequential access gives short seeks.	No defragmentation; free space can still fragment.

Decision	Reason	Trade-off or limitation
Write-through metadata	Bitmap and superblock are on disk before a block is used; simpler than a cache.	More I/O per allocation; no crash consistency without a journal.
Start dispatcher in `op_init`	Threads do not survive `fork()` during daemonization.	The image is opened twice (validation, then real mount).
Seek = $\|\Delta\text{block}\|$	Matches the textbook model; independent of the host device.	Does not model rotation, SSD behaviour or device caches.

6. IMPLEMENTATION DETAILS

6.1 Implementation Overview

SchedFS is written in C11 and builds three executables from one source tree (Table 12): the file system `schedfs`, the formatter `mkfs.schedfs` and the benchmark `schedfs-bench`. The headers in `include/` define every interface, so each `.c` file depends only on the layer directly below it. The execution flow follows Fig. 3: `main()` in `fuse_main.c` validates the options and the image, `fuse_main()` hands control to libfuse, and from then on the program is driven by kernel callbacks on libfuse worker threads plus one dispatcher thread.

Table 12 Source organisation

Directory	Files	Role
`include/`	8 headers	on-disk format (`layout.h`) and the interface of every module
`src/fuse/`	`fuse_main.c`	option parsing, mount lifecycle, 16 FUSE callbacks
`src/fs/`	`filesystem.c`	inodes, directories, paths, bitmap allocation, block mapping
`src/sync/`	`request_queue.c`	request life cycle and the bounded queue
`src/sched/`	`scheduler.c`	FCFS, SSTF, SCAN and Adaptive selection
`src/storage/`	`virtual_disk.c`	image file, dispatcher thread, `pread`/`pwrite`
`src/bench/`	`benchmark_main.c`, `metrics.c`	trace replay, metric accumulation
`src/utils/`	`logger.c`	thread-safe levelled logging
`mkfs/`	`mkfs.c`	image formatter
`tests/`	2 C tests, 1 shell script	scheduler, core and mounted system tests
`scripts/`	2 Python scripts	graphs, workload generation
`docs/`	`index.html` and 3 Markdown files	interactive demo, architecture notes

The main data structures are listed in Table 13. The three on-disk structures are packed,

and their sizes are checked at compile time with `_Static_assert` (Listing 1), so a change that alters the format fails to compile.

Table 13 Principal data structures

Structure	Defined in	Key fields and purpose
`sfs_superblock_t`	`layout.h`	magic, geometry, free counts, region offsets; 128 bytes at block 0
`sfs_inode_t`	`layout.h`	mode, links, owner, size, times, `direct[12]`, `indirect`; 128 bytes
`sfs_dirent_t`	`layout.h`	inode number and a 60-byte name; 64 entries per block
`disk_request_t`	`request.h`	ID, operation, block, buffer, arrival/service/completion times, mutex, condition variable, `completed`, result
`request_queue_t`	`request_queue.h`	array of request pointers, count, capacity, `stopping`, mutex, `not_empty`, `not_full`
`scheduler_t`	`scheduler.h`	policy, total blocks, head, SCAN direction, α, β
`virtual_disk_t`	`virtual_disk.h`	file descriptor, queue, scheduler, metrics, dispatcher thread
`sfs_t`	`filesystem.h`	disk, in-memory superblock and bitmap, global mutexes, 257 inode rwlocks, 257 directory mutexes

6.2 Module-Wise Implementation

M1 Image formatter. `main()` in `mkfs/mkfs.c` validates the size with `parse_size()`, computes the layout (Listing 2), and writes each region with `write_full()`, which repeats `pwrite` until every byte is written and retries on `EINTR`. Errors are reported with `perror`; any failure after the file is created jumps to a `fail:` label that closes and unlinks the image, so a half-formatted image is never left behind.

M2 FUSE adapter. The callbacks are thin. Each one converts its arguments and returns the `sfs_*` result, which is already a negative `errno` as libfuse expects. The `init` and `destroy` callbacks (Listing 3) own the file-system instance. If the image cannot be mounted inside the daemon, `op_init` logs the error and calls `fuse_exit()`, so the process stops cleanly instead of serving a broken mount.

M3 Inode and directory layer. Public operations follow one pattern: resolve the path, lock the inode in the right mode, read the inode, perform block operations, write the inode back and unlock. `inode_write` (Listing 4) is the read–modify–write that needed the inode-table mutex. `sfs_rename` (Listing 6) shows the error-handling style for multi-step metadata changes: every step that succeeded is undone if a later step fails.

M4 Block allocation and mapping. `bmap` (Listing 5) returns block 0 for an unallocated logical block when `create` is false, which is how holes read as zeros, and allocates on demand when `create` is true. `free_inode_blocks` releases direct blocks, then indirect entries, and finally the indirect block itself once it is empty.

M5 Request queue. Listing 7 shows the two queue operations and the completion wait. Every `pthread_cond_wait` sits in a `while` loop that re-tests its predicate. Removal uses `memmove`, so the array keeps arrival order, which FCFS relies on.

M6 Scheduler. Listing 8 is the complete selection function. `older()` gives the deterministic tie-break. SCAN falls back to the nearest request when nothing lies in the current direction; the dispatcher notices that the head has turned back and flips the direction.

M7 Virtual disk and dispatcher. `transfer()` (Listing 9) rejects out-of-range blocks with `EINVAL` before queuing anything. The dispatcher treats a short transfer as `EIO` and passes the error code back through the request, so a failure reaches the application as a normal `errno`.

M8 Metrics and logging. `metrics_record` (Listing 10) accumulates the sum and sum of squares of the waiting times, so the mean and standard deviation can be computed without storing every sample. The logger serialises writes with a mutex and filters DEBUG entries unless `--verbose` is set.

M9 Benchmark. The replay loop (Listing 11) charges SCAN for travelling to the disk edge before it reverses (block 0 when moving down, block 4095 when moving up), which is what distinguishes SCAN's totals from LOOK's.

M10 Tests. `test_scheduler` (Listing 12) asserts both the exact service order and the total head movement. `system.sh` adds a regression test for background mounting (Listing 13); every step runs under `timeout`, so a hang fails the test instead of blocking it.

6.3 Source Code

The listings below are copied verbatim from the repository, by line range, with the original line numbers. Complete files for the scheduling path are in Appendix A.

Listing 1 On-disk structures with compile-time size checks (`include/layout.h`)

```
typedef struct __attribute__((packed)) {
    uint32_t magic, version, block_size, total_blocks;
    uint32_t free_blocks, total_inodes, free_inodes;
    uint32_t bitmap_start, bitmap_blocks, inode_start, inode_blocks;
    uint32_t data_start, root_inode, checksum;
    uint8_t reserved[72];
} sfs_superblock_t;

typedef struct __attribute__((packed)) {
    uint16_t mode, links;
    uint32_t uid, gid;
    uint64_t size, atime, mtime, ctime;
    uint32_t direct[SFS_DIRECT_BLOCKS];
    uint32_t indirect, generation;
    uint8_t reserved[28];
} sfs_inode_t;

typedef struct __attribute__((packed)) {
    uint32_t ino;
    char name[60];
} sfs_dirent_t;

_Static_assert(sizeof(sfs_superblock_t) == 128, "superblock header must be 128
    ↪ bytes");
_Static_assert(sizeof(sfs_inode_t) == 128, "inode must be 128 bytes");
_Static_assert(sizeof(sfs_dirent_t) == 64, "directory entry must be 64 bytes");
```

The three `_Static_assert` lines tie the C structures to the on-disk format: 32 inodes and 64 directory entries fit exactly in one 4 KiB block.

Listing 2 Layout computation and safe creation of the image (`mkfs/mkfs.c`)

```
    uint64_t blocks64 = bytes / SFS_BLOCK_SIZE;
    if (blocks64 < SFS_MIN_BLOCKS || blocks64 > UINT32_MAX) {
        fprintf(stderr, "unsupported image size\n");
        return 2;
    }
    uint32_t blocks = (uint32_t)blocks64;
    uint32_t bitmap_blocks = (blocks + SFS_BLOCK_SIZE * 8u - 1u) / (
    ↪ SFS_BLOCK_SIZE * 8u),
             inode_blocks =
                 (SFS_DEFAULT_INODES * (uint32_t)sizeof(sfs_inode_t) +
    ↪ SFS_BLOCK_SIZE - 1u) /
                 SFS_BLOCK_SIZE;
    uint32_t data_start = 1u + bitmap_blocks + inode_blocks;
    if (data_start >= blocks) {
        fprintf(stderr, "image too small for metadata\n");
        return 2;
    }
    int fd = open(argv[1], O_CREAT | O_EXCL | O_RDWR | O_CLOEXEC, 0644);
    if (fd < 0) {
        perror("open (remove an old image explicitly before formatting)");
        return 1;
```

For 4,096 blocks this gives one bitmap block, eight inode blocks and `data_start` = 10, which matches the output in Fig. 20.

Listing 3 Starting the file system inside the FUSE daemon (`src/fuse/fuse_main.c`)

```

/*
 * Runs after fuse_main() has daemonized. The dispatcher thread must be created
 * here: threads started before the fork() would not exist in the daemon.
 */
static void *op_init(struct fuse_conn_info *conn, struct fuse_config *cfg) {
    (void)conn;
    (void)cfg;
    int rc = sfs_mount_image(&g_fs, g_cfg.image, g_cfg.policy, g_cfg.alpha, g_cfg.
    ↪ beta,
                             g_cfg.verbose);
    if (rc) {
        sfs_log(LOG_ERROR, "cannot start file system: %s", strerror(-rc));
        fuse_exit(fuse_get_context()->fuse);
    } else
        g_mounted = true;
    return NULL;
}

static void op_destroy(void *private_data) {
    (void)private_data;
    if (g_mounted)
        sfs_unmount_image(&g_fs);
    g_mounted = false;
}
```

Listing 4 Inode update protected by the inode-table mutex (`src/fs/filesystem.c`)

```
static int inode_write(sfs_t *fs, uint32_t ino, const sfs_inode_t *in) {
    uint32_t per = SFS_BLOCK_SIZE / (uint32_t)sizeof(*in), idx = ino - 1u;
    uint8_t block[SFS_BLOCK_SIZE];
    pthread_mutex_lock(&fs->inode_table_lock);
    int rc = block_read(fs, fs->sb.inode_start + idx / per, block);
    if (!rc) {
        memcpy(block + (idx % per) * sizeof(*in), in, sizeof(*in));
        rc = block_write(fs, fs->sb.inode_start + idx / per, block);
    }
    pthread_mutex_unlock(&fs->inode_table_lock);
    return rc;
}
```

Listing 5 Logical-to-physical block mapping with on-demand allocation (`src/fs/filesystem.c`)

```
static int bmap(sfs_t *fs, sfs_inode_t *n, uint32_t logical, bool create,
    ↪ uint32_t *physical) {
    if (logical >= SFS_DIRECT_BLOCKS + SFS_INDIRECT_BLOCKS)
        return -EFBIG;
    if (logical < SFS_DIRECT_BLOCKS) {
        if (!n->direct[logical] && create) {
            uint32_t allocated = 0;
            uint32_t hint = logical ? n->direct[logical - 1] : fs->sb.data_start;
            int rc = alloc_block(fs, hint, &allocated);
            if (rc)
                return rc;
            n->direct[logical] = allocated;
        }
```

```
        *physical = n->direct[logical];
        return 0;
    }
    if (!n->indirect) {
        if (!create) {
            *physical = 0;
            return 0;
        }
        uint32_t allocated = 0;
        int rc = alloc_block(fs, fs->sb.data_start, &allocated);
        if (rc)
            return rc;
        n->indirect = allocated;
    }
    uint32_t ptrs[SFS_INDIRECT_BLOCKS];
    int rc = block_read(fs, n->indirect, ptrs);
    if (rc)
        return rc;
    uint32_t idx = logical - SFS_DIRECT_BLOCKS;
    if (!ptrs[idx] && create) {
        rc = alloc_block(fs, n->indirect, &ptrs[idx]);
        if (rc)
            return rc;
        rc = block_write(fs, n->indirect, ptrs);
        if (rc)
            return rc;
    }
    *physical = ptrs[idx];
    return 0;
}
```

Lines 191–192 pass the previous direct block as the allocation hint, which keeps a file's blocks adjacent (Fig. 10).

Listing 6 Rename with roll-back of completed steps on failure (`src/fs/filesystem.c`)

```
int sfs_rename(sfs_t *fs, const char *from, const char *to) {
    uint32_t fp = 0, tp = 0, ino = 0;
    char fn[60], tn[60];
    bool target_added = false;
    int rc = parent_name(fs, from, &fp, fn);
    if (!rc)
        rc = parent_name(fs, to, &tp, tn);
    if (!rc)
        rc = dir_find(fs, fp, fn, &ino, NULL);
    if (!rc && !dir_find(fs, tp, tn, NULL, NULL))
        return -EEXIST;
    if (!rc) {
        rc = dir_add(fs, tp, tn, ino);
        target_added = !rc;
    }
    if (!rc && fp != tp) {
        sfs_inode_t n;
        if (!inode_read(fs, ino, &n) && S_ISDIR(n.mode)) {
            uint32_t old_parent = 0;
            rc = dir_remove(fs, ino, "..", &old_parent);
            if (!rc)
                rc = dir_add(fs, ino, "..", tp);
```

```
            if (rc && old_parent)
                dir_add(fs, ino, "..", old_parent);
        }
    }
    if (!rc) {
        rc = dir_remove(fs, fp, fn, NULL);
        if (rc)
            dir_remove(fs, tp, tn, NULL);
    } else if (target_added)
        dir_remove(fs, tp, tn, NULL);
    return rc;
}
```

Listing 7 Completion wait and the bounded producer–consumer queue (`src/sync/request_queue.c`)

```
int request_wait(disk_request_t *r) {
    pthread_mutex_lock(&r->mutex);
    while (!r->completed)
        pthread_cond_wait(&r->completed_cv, &r->mutex);
    int result = r->error ? -r->error : (r->result < 0 ? -EIO : 0);
    pthread_mutex_unlock(&r->mutex);
    return result;
}

int rq_init(request_queue_t *q, size_t capacity) {
    memset(q, 0, sizeof(*q));
    if (!capacity)
        return -EINVAL;
    q->items = calloc(capacity, sizeof(*q->items));
    if (!q->items)
        return -ENOMEM;
    q->capacity = capacity;
    pthread_mutex_init(&q->mutex, NULL);
    pthread_cond_init(&q->not_empty, NULL);
    pthread_cond_init(&q->not_full, NULL);
    return 0;
}

int rq_submit(request_queue_t *q, disk_request_t *r) {
    pthread_mutex_lock(&q->mutex);
    while (q->count == q->capacity && !q->stopping)
        pthread_cond_wait(&q->not_full, &q->mutex);
    if (q->stopping) {
        pthread_mutex_unlock(&q->mutex);
        return -ESHUTDOWN;
    }
    q->items[q->count++] = r;
    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->mutex);
    return 0;
}

disk_request_t *rq_take(request_queue_t *q, const scheduler_t *scheduler) {
    pthread_mutex_lock(&q->mutex);
    while (!q->count && !q->stopping)
        pthread_cond_wait(&q->not_empty, &q->mutex);
    if (!q->count) {
```

```
43          pthread_mutex_unlock(&q->mutex);
44          return NULL;
45      }
46      struct timespec now;
47      clock_gettime(CLOCK_MONOTONIC, &now);
48      size_t index = scheduler_pick(scheduler, q->items, q->count, now);
49      disk_request_t *r = q->items[index];
50      memmove(&q->items[index], &q->items[index + 1], (q->count - index - 1) *
        ↪ sizeof(*q->items));
51      --q->count;
52      pthread_cond_signal(&q->not_full);
53      pthread_mutex_unlock(&q->mutex);
54      return r;
55  }
```

The scheduler runs at line 86 while the queue mutex is held, which is safe because `scheduler_pick` never blocks. The I/O itself happens after `rq_take` has released the mutex.

Listing 8 Request selection for all four policies (`src/sched/scheduler.c`)

```
1   static int older(const disk_request_t *a, const disk_request_t *b) {
2       if (a->arrival.tv_sec != b->arrival.tv_sec)
3           return a->arrival.tv_sec < b->arrival.tv_sec;
4       if (a->arrival.tv_nsec != b->arrival.tv_nsec)
5           return a->arrival.tv_nsec < b->arrival.tv_nsec;
6       return a->id < b->id;
7   }
8
9   size_t scheduler_pick(const scheduler_t *s, disk_request_t *const *items, size_t
        ↪ count,
10                        struct timespec now) {
11      size_t best = 0;
12      if (count < 2 || s->policy == SCHED_FCFS) {
13          for (size_t i = 1; i < count; ++i)
14              if (older(items[i], items[best]))
15                  best = i;
16          return best;
17      }
18      if (s->policy == SCHED_SSTF) {
19          for (size_t i = 1; i < count; ++i) {
20              uint32_t di = distance(s->head, items[i]->block);
21              uint32_t db = distance(s->head, items[best]->block);
22              if (di < db || (di == db && older(items[i], items[best])))
23                  best = i;
24          }
25          return best;
26      }
27      if (s->policy == SCHED_SCAN) {
28          bool found = false;
29          for (size_t i = 0; i < count; ++i) {
30              bool direction_ok =
31                  s->scan_direction > 0 ? items[i]->block >= s->head : items[i]->
        ↪ block <= s->head;
32              if (!direction_ok)
33                  continue;
34              if (!found ||
```

```
35                  distance(s->head, items[i]->block) < distance(s->head, items[best
    ↪ ]->block) ||
36                  (distance(s->head, items[i]->block) == distance(s->head, items[
    ↪ best]->block) &&
37                   older(items[i], items[best]))) {
38                  best = i;
39                  found = true;
40              }
41          }
42          if (found)
43              return best;
44          for (size_t i = 1; i < count; ++i) {
45              if (distance(s->head, items[i]->block) < distance(s->head, items[best
    ↪ ]->block) ||
46                  (distance(s->head, items[i]->block) == distance(s->head, items[
    ↪ best]->block) &&
47                   older(items[i], items[best])))
48                  best = i;
49          }
50          return best;
51      }
52      double max_wait = 0.0;
53      for (size_t i = 0; i < count; ++i) {
54          double w = elapsed_ms(items[i]->arrival, now);
55          if (w > max_wait)
56              max_wait = w;
57      }
58      double best_score = INFINITY;
59      for (size_t i = 0; i < count; ++i) {
60          double dn = (double)distance(s->head, items[i]->block) / (double)s->
    ↪ total_blocks;
61          double wn = max_wait > 0.0 ? elapsed_ms(items[i]->arrival, now) /
    ↪ max_wait : 0.0;
62          double score = s->alpha * dn - s->beta * wn;
63          if (score < best_score - 1e-12 ||
64              (fabs(score - best_score) <= 1e-12 && older(items[i], items[best]))) {
    ↪
65              best = i;
66              best_score = score;
67          }
68      }
69      return best;
70  }
```

Lines 88–105 implement Equation (1). The 10^{-12} tolerance makes equal scores compare as equal despite floating-point rounding, so the tie-break still applies.

Listing 9 Dispatcher thread and the synchronous transfer used by the file system (`src/storage/virtual_disk.c`)

```
static void *dispatcher(void *arg) {
    virtual_disk_t *d = arg;
    while (true) {
        disk_request_t *r = rq_take(&d->queue, &d->scheduler);
        if (!r)
            break;
        clock_gettime(CLOCK_MONOTONIC, &r->service_start);
        uint32_t from = d->scheduler.head;
        r->result = full_pio(d->fd, r->buffer, SFS_BLOCK_SIZE, (off_t)r->block *
    ↪ SFS_BLOCK_SIZE,
                             r->op == RQ_WRITE);
        r->error = r->result < 0 ? (int)-r->result : 0;
        clock_gettime(CLOCK_MONOTONIC, &r->completion);
        metrics_record(&d->metrics, from, r->block, diff_ms(r->arrival, r->
    ↪ service_start));
        if (d->scheduler.policy == SCHED_SCAN) {
            if (d->scheduler.scan_direction > 0 && r->block < from)
                d->scheduler.scan_direction = -1;
            else if (d->scheduler.scan_direction < 0 && r->block > from)
                d->scheduler.scan_direction = 1;
        }
        d->scheduler.head = r->block;
        pthread_mutex_lock(&r->mutex);
        r->completed = true;
        pthread_cond_signal(&r->completed_cv);
        pthread_mutex_unlock(&r->mutex);
    }
    return NULL;
}

int vdisk_open(virtual_disk_t *d, const char *path, uint32_t blocks,
    ↪ sched_policy_t policy,
               double alpha, double beta, size_t capacity) {
    memset(d, 0, sizeof(*d));
    d->fd = open(path, O_RDWR | O_CLOEXEC);
    if (d->fd < 0)
        return -errno;
    d->blocks = blocks;
    d->scheduler = (scheduler_t){policy, blocks, 0, 1, alpha, beta};
    int rc = rq_init(&d->queue, capacity);
    if (rc) {
        close(d->fd);
        return rc;
    }
    metrics_init(&d->metrics);
    rc = pthread_create(&d->thread, NULL, dispatcher, d);
    if (rc) {
        rq_destroy(&d->queue);
        close(d->fd);
        return -rc;
    }
    d->running = 1;
    return 0;
}

static int transfer(virtual_disk_t *d, uint32_t block, void *buf, rq_op_t op) {
```

```
    if (block >= d->blocks)
        return -EINVAL;
    disk_request_t r;
    int rc = request_init(&r, op, block, buf);
    if (rc)
        return rc;
    rc = rq_submit(&d->queue, &r);
    if (!rc)
        rc = request_wait(&r);
    request_destroy(&r);
    return rc;
}
```

Listing 10 Thread-safe metric accumulation (`src/bench/metrics.c`)

```
void metrics_record(sfs_metrics_t *m, uint32_t from, uint32_t to, double wait_ms)
    ↪ {
    pthread_mutex_lock(&m->mutex);
    m->requests++;
    m->total_seek += from > to ? from - to : to - from;
    m->wait_sum_ms += wait_ms;
    m->wait_sq_sum_ms += wait_ms * wait_ms;
    if (wait_ms > m->wait_max_ms)
        m->wait_max_ms = wait_ms;
    pthread_mutex_unlock(&m->mutex);
}
```

Listing 11 Replay loop and SCAN edge accounting (`src/bench/benchmark_main.c`)

```
    scheduler_t s = {p, 4096, 53, -1, .7, .3};
    uint64_t seek = 0;
    double sum = 0, max = 0, sq = 0;
    size_t n = w->count;
    now_time(&now);
    while (n) {
        size_t k = scheduler_pick(&s, pending, n, now);
        uint32_t next = pending[k]->block;
        uint64_t move = s.head > next ? s.head - next : next - s.head;
        if (p == SCHED_SCAN && s.scan_direction < 0 && next > s.head) {
            move = (uint64_t)s.head + next;
            s.scan_direction = 1;
        } else if (p == SCHED_SCAN && s.scan_direction > 0 && next < s.head) {
            move = (uint64_t)(4095 - s.head) + (4095 - next);
            s.scan_direction = -1;
        }
        seek += move;
        s.head = next;
        double wait = ms(pending[k]->arrival, now);
        if (wait < 0)
            wait = 0;
        sum += wait;
        sq += wait * wait;
```

Listing 12 Textbook-trace unit test (`tests/test_scheduler.c`)

```
static uint64_t check(sched_policy_t p, const uint32_t *expected) {
    uint32_t blocks[] = {98, 183, 37, 122, 14, 124, 65, 67};
    disk_request_t r[8];
    disk_request_t *items[8];
    struct timespec now = {1, 0};
    for (size_t i = 0; i < 8; ++i) {
        memset(&r[i], 0, sizeof(r[i]));
        r[i].id = i + 1;
        r[i].block = blocks[i];
        r[i].arrival.tv_nsec = (long)i;
        items[i] = &r[i];
    }
    scheduler_t s = {p, 200, 53, -1, .7, .3};
    size_t n = 8;
    uint64_t movement = 0;
    for (size_t step = 0; step < 8; ++step) {
        size_t k = scheduler_pick(&s, items, n, now);
        assert(items[k]->block == expected[step]);
        uint32_t old = s.head, next = items[k]->block;
        if (p == SCHED_SCAN && s.scan_direction < 0 && next > old) {
            movement += (uint64_t)old + next;
            s.scan_direction = 1;
        } else
            movement += old > next ? old - next : next - old;
        s.head = next;
        memmove(&items[k], &items[k + 1], (--n - k) * sizeof(*items));
    }
    return movement;
}

int main(void) {
    uint32_t f[] = {98, 183, 37, 122, 14, 124, 65, 67}, s[] = {65, 67, 37, 14, 98,
    ↪  122, 124, 183},
             scan[] = {37, 14, 65, 67, 98, 122, 124, 183};
    assert(check(SCHED_FCFS, f) == 640);
    assert(check(SCHED_SSTF, s) == 236);
    assert(check(SCHED_SCAN, scan) == 236);
    puts("scheduler tests passed");
    return 0;
```

Listing 13 Background-mount regression test (`tests/system.sh`)

```
# Background (daemonized) mount, as documented in the README: no -f, relative
    ↪ image path.
cp "$image" "$tmp/bg.img"
(cd "$tmp" && "$OLDPWD/schedfs" bg.img mnt --sched=scan >/dev/null)
for _ in {1..50}; do mountpoint -q "$mnt" && break; sleep .1; done
mountpoint -q "$mnt" || { echo "background mount failed"; exit 1; }
timeout 10 mkdir "$mnt/bg" || { echo "background mount does not serve I/O"; exit
    ↪ 1; }
timeout 10 sh -c "echo persisted > '$mnt/bg/x.txt'"
test "$(timeout 10 cat "$mnt/bg/x.txt")" = persisted
fusermount3 -u "$mnt"
for _ in {1..50}; do mountpoint -q "$mnt" || break; sleep .1; done
(cd "$tmp" && "$OLDPWD/schedfs" bg.img mnt --sched=fcfs >/dev/null)
for _ in {1..50}; do mountpoint -q "$mnt" && break; sleep .1; done
test "$(timeout 10 cat "$mnt/bg/x.txt")" = persisted || { echo "data lost across
    ↪ remount"; exit 1; }
fusermount3 -u "$mnt"
echo "FUSE system tests passed"
```

6.4 Code Contribution Summary

Counting method. Lines were counted with `report/tools/count_loc.py`, which is included in the repository with this report. It reads every C source and header, shell script, Python script and the Makefile in the repository, and it skips blank lines and comment-only lines. C block comments are removed before counting. Generated files, third-party code, `docs/`, `results/`, this `report/` folder and the 655-line HTML demo page are excluded. The result is **2,103 qualifying lines** (2,369 physical lines). Table 14 gives the per-file counts.

Table 14 Qualifying lines of code per file

File	Qualifying	Physical	Module
`src/fs/filesystem.c`	773	814	M3, M4
`src/fuse/fuse_main.c`	199	234	M2
`mkfs/mkfs.c`	145	154	M1
`src/bench/benchmark_main.c`	130	142	M9
`src/storage/virtual_disk.c`	103	116	M7
`src/sched/scheduler.c`	95	106	M6
`src/sync/request_queue.c`	94	108	M5
`tests/test_core.c`	91	100	M10
`scripts/plot_results.py`	88	108	M9
`tests/test_scheduler.c`	42	49	M10
`src/utils/logger.c`	40	49	M8
`Makefile`	35	36	M10
`include/filesystem.h`	35	43	M3, M4

File	Qualifying	Physical	Module
include/layout.h	35	47	M1
include/request.h	35	44	M5
tests/system.sh	32	34	M10
scripts/generate_workload.py	28	38	M9
src/bench/metrics.c	21	29	M8
include/virtual_disk.h	21	28	M7
include/request_queue.h	20	27	M5
include/scheduler.h	18	26	M6
include/metrics.h	15	22	M8
include/logger.h	8	15	M8
Total	**2,103**	**2,369**	

Table 15 shows the modules and files each team member implemented, with the qualifying lines of code in them.

Table 15 Code contribution summary

Member	Modules and files	LOC
Saanvi Grover	M1, M2, M8 logger: fuse_main.c, mkfs.c, layout.h, filesystem.h, logger.c/.h, system.sh	494
Prankit Goel	M5, M6, M7: request_queue.c, scheduler.c, virtual_disk.c, their headers, test_scheduler.c	428
Aditya Sharma	M3, M4: filesystem.c	773
Aaradhya Shankar Kulashri	M8 metrics, M9, M10: benchmark_main.c, metrics.c/.h, both Python scripts, test_core.c, Makefile	408
Total	all modules	**2,103**

6.5 Implementation Screenshots

All terminal figures in this report come from one run of the repository at commit a7e489c in an ubuntu:24.04 container with /dev/fuse, executed as the unprivileged user student. Each figure is drawn from the captured terminal text without editing; the raw logs are kept in figures/screenshots/raw_logs/. The browser figures are screenshots of the live demo page.

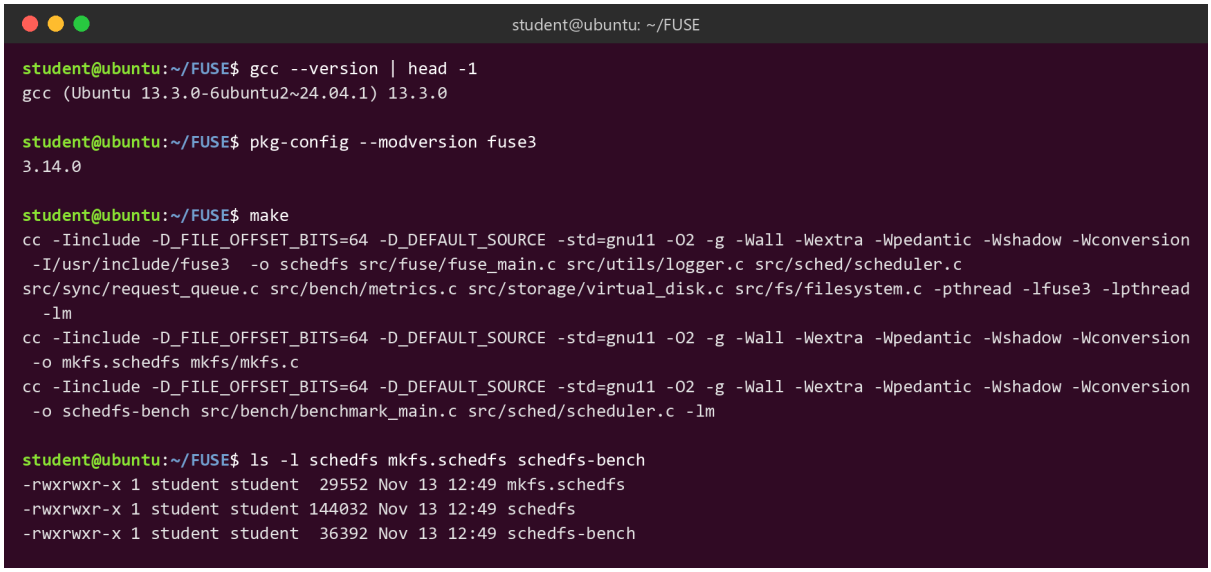


Fig. 19 Building the three executables with strict warning flags; the compiler emits no warnings.

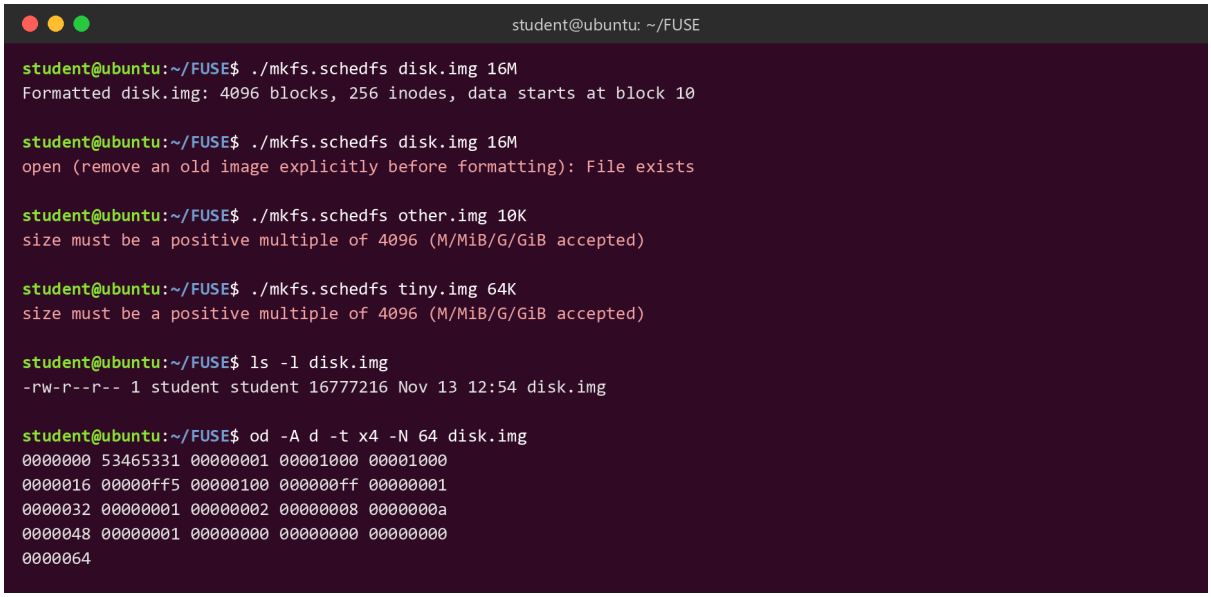


Fig. 20 Formatting an image, refusing to overwrite it, rejecting invalid sizes, and dumping the superblock.

Fig. 20 shows FR1 at work. The second `mkfs` is refused because the image exists. The sizes `10K` and `64K` are both rejected because the formatter accepts only the suffixes M, MiB, G and GiB; the message says so. The `od` dump is decoded in Table 8.

```
student@ubuntu: ~/FUSE
student@ubuntu:~/FUSE$ mkdir -p mnt

student@ubuntu:~/FUSE$ ./schedfs disk.img mnt --sched=sstf
SchedFS: 4096 blocks x 4096 bytes, 256 inodes, scheduler=sstf

student@ubuntu:~/FUSE$ mount | grep schedfs
/home/student/FUSE/schedfs on /home/student/FUSE/mnt type fuse.schedfs
(rw,nosuid,nodev,relatime,user_id=1001,group_id=1001)

student@ubuntu:~/FUSE$ mkdir mnt/project

student@ubuntu:~/FUSE$ echo 'Operating Systems' > mnt/project/demo.txt

student@ubuntu:~/FUSE$ cat mnt/project/demo.txt
Operating Systems

student@ubuntu:~/FUSE$ cp mnt/project/demo.txt mnt/project/copy.txt

student@ubuntu:~/FUSE$ head -c 100000 /dev/urandom > /tmp/big.bin && cp /tmp/big.bin mnt/project/big.bin && cmp
/tmp/big.bin mnt/project/big.bin && echo identical
identical

student@ubuntu:~/FUSE$ ls -l mnt/project
total 99
-rw-rw-r-- 1 student student 100000 Nov 13 12:49 big.bin
-rw-rw-r-- 1 student student     18 Nov 13 12:49 copy.txt
-rw-rw-r-- 1 student student     18 Nov 13 12:49 demo.txt

student@ubuntu:~/FUSE$ stat mnt/project/copy.txt
  File: mnt/project/copy.txt
  Size: 18            Blocks: 1          IO Block: 4096   regular file
Device: 0,93    Inode: 4           Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1001/ student)   Gid: ( 1001/ student)
Access: 2026-11-13 12:49:54.000000000 +0000
Modify: 2026-11-13 12:49:54.000000000 +0000
Change: 2026-11-13 12:49:54.000000000 +0000
 Birth: -
```

Fig. 21 Mounting under SSTF and using the file system with ordinary commands.

In Fig. 21 the kernel lists the mount as type `fuse.schedfs`. The 100,000-byte file needs 25 blocks, more than the 12 direct pointers, so it exercises the single-indirect block; `cmp` confirms that it reads back identical to the source.

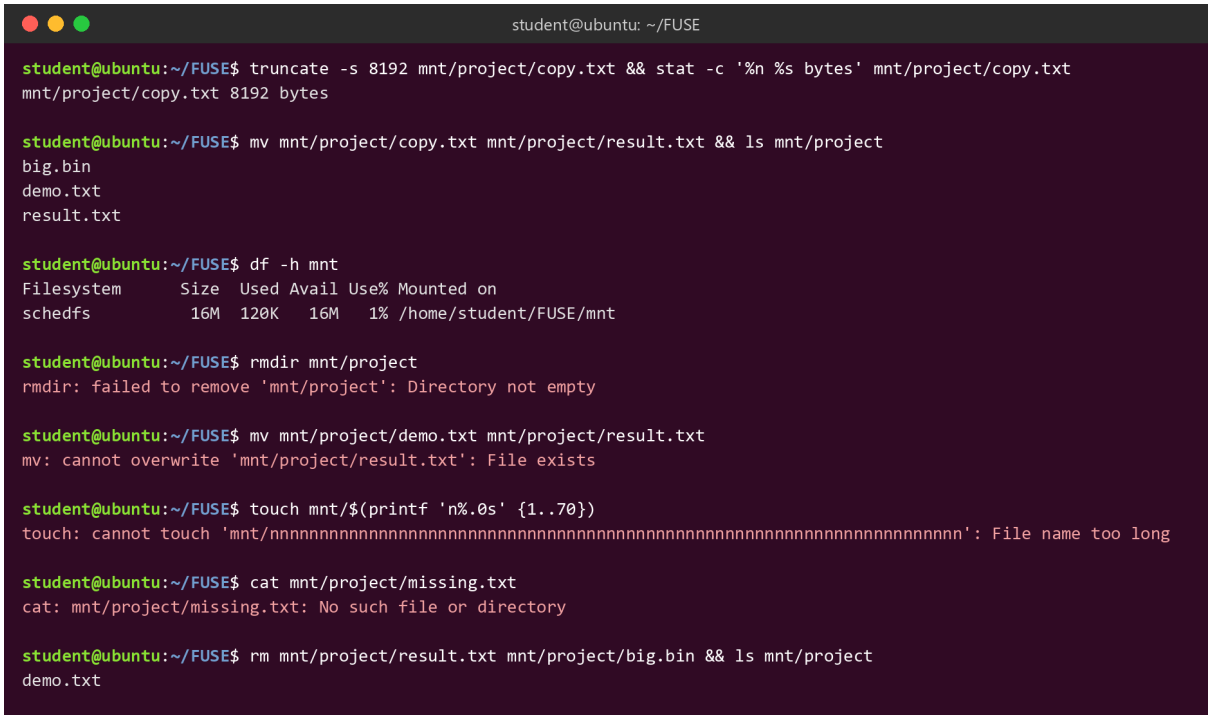


Fig. 22 Truncate, rename, capacity reporting and error handling on the mounted file system.

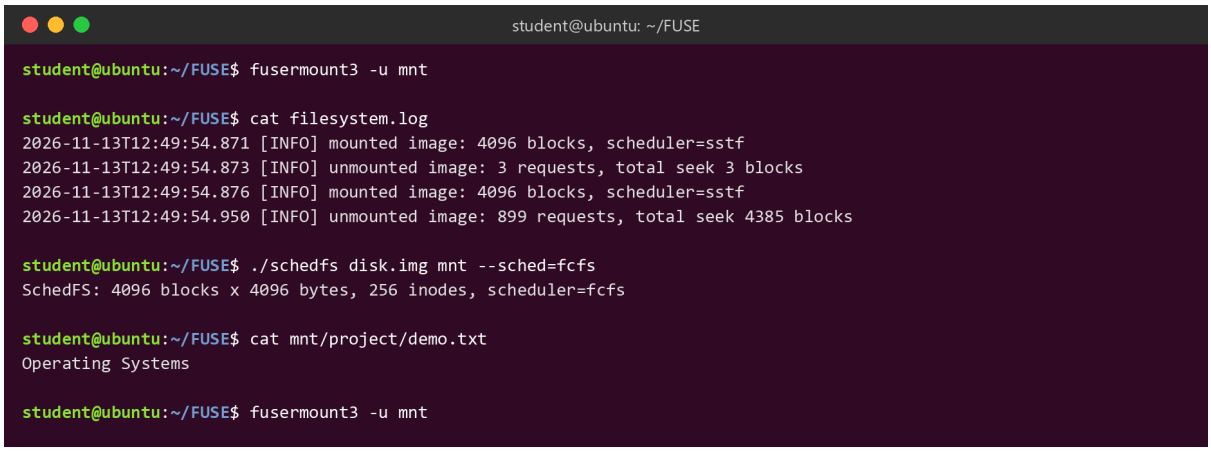


Fig. 23 Unmount summary in `filesystem.log`, and data persisting after a remount under FCFS.

Each mount writes two log lines (Fig. 23). The first pair, with 3 requests, comes from the validation mount in `main()`; the second pair is the real session, which served 899 block requests with a total head movement of 4,385 blocks.

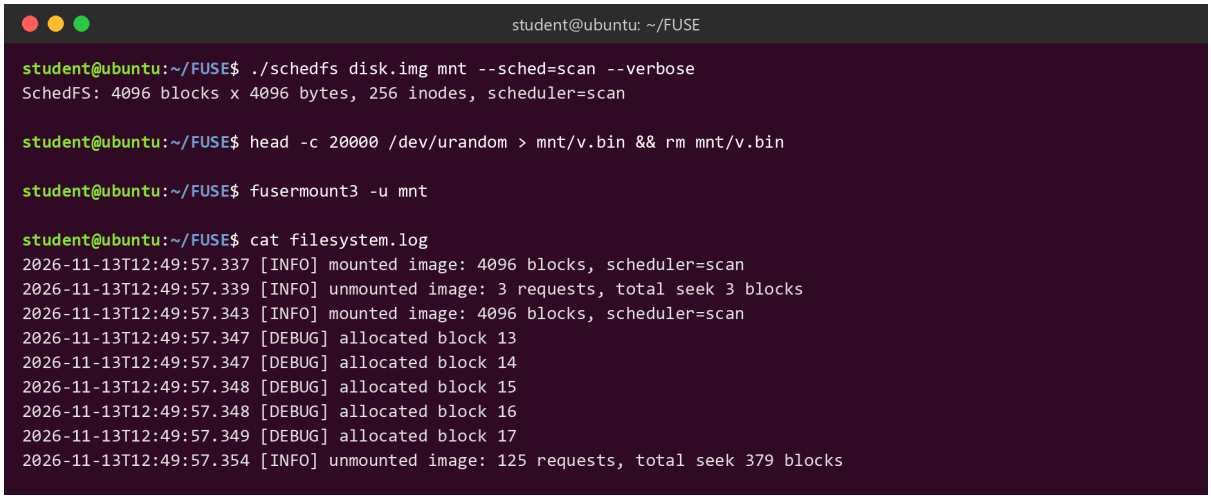


Fig. 24 Verbose mode under SCAN: DEBUG entries show five consecutive data blocks allocated for a 20,000-byte file.

The allocations in Fig. 24 are consecutive (blocks 13–17) because of the hint-based allocator described in Fig. 10.

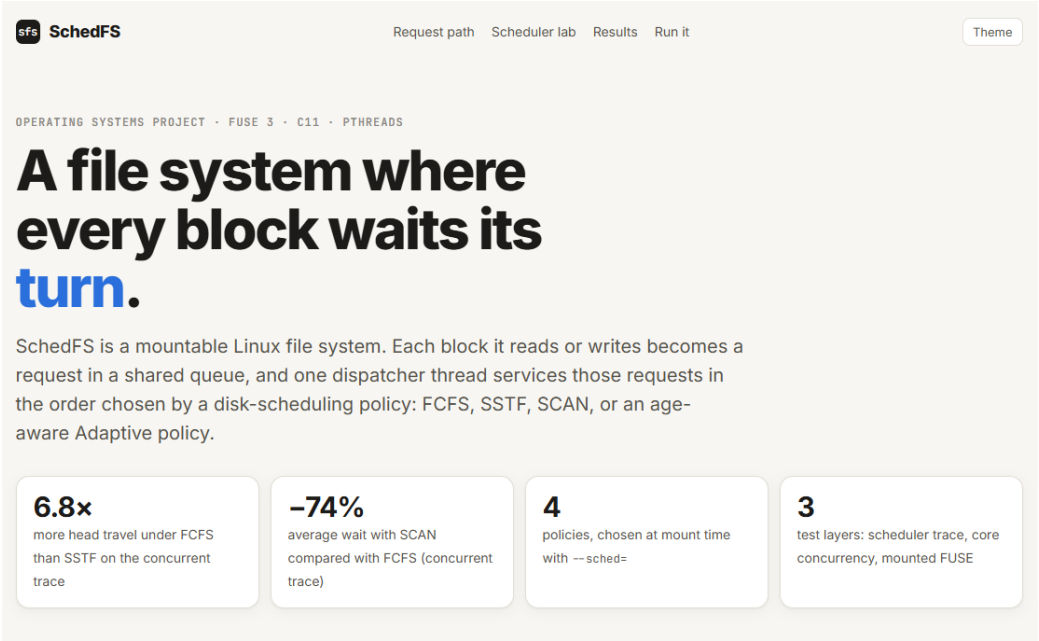


Fig. 25 Interactive demo page (GitHub Pages): overview of the project.

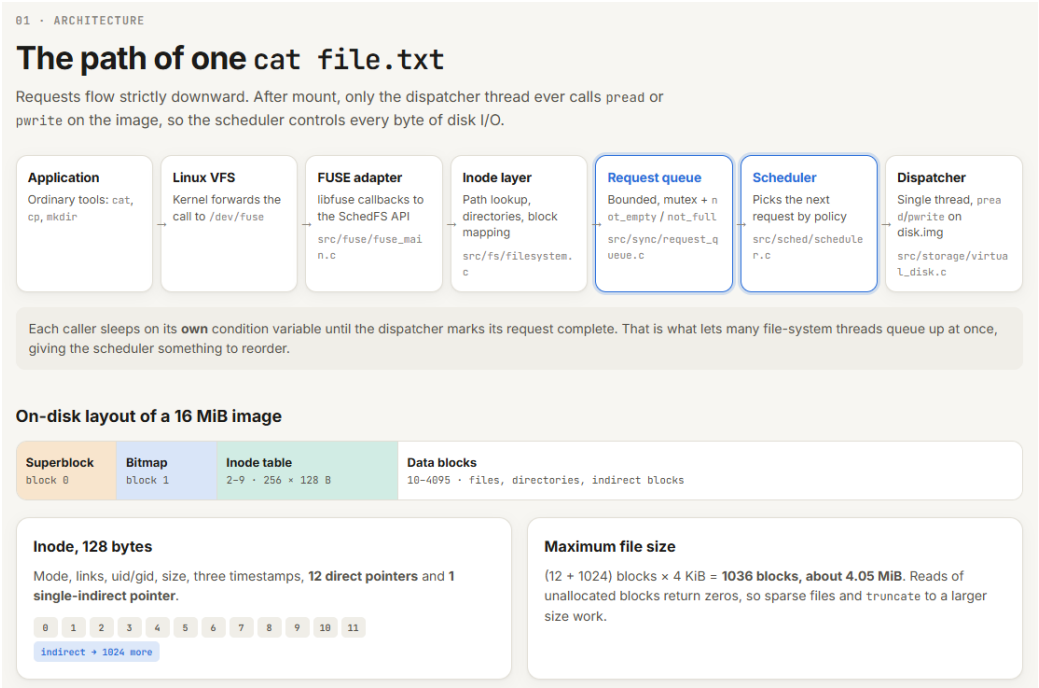


Fig. 26 Demo page: the request path and the on-disk layout, linked to the source files.

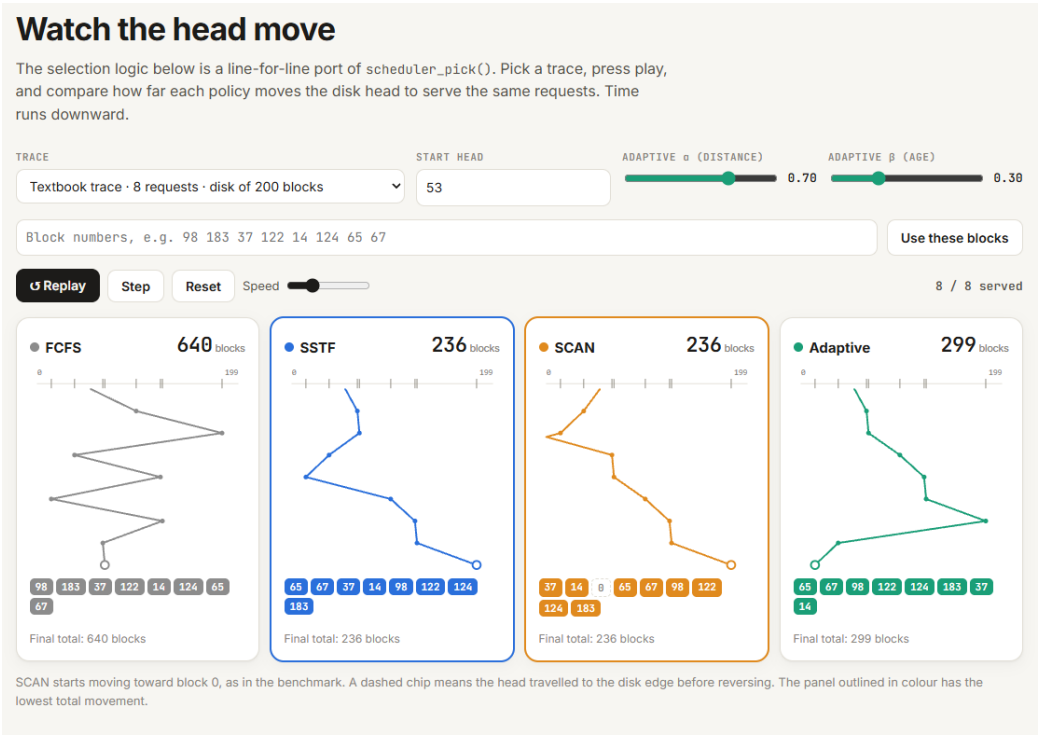


Fig. 27 Scheduler lab after playing the textbook trace: FCFS 640, SSTF 236, SCAN 236 and Adaptive 299 blocks.

The lab in Fig. 27 runs a JavaScript port of `scheduler_pick()`. Its FCFS, SSTF and SCAN totals and service orders match the C unit test exactly. Its Adaptive total (299) uses a modelled clock in the browser, so it is shown for illustration only and is not one of the tested values.

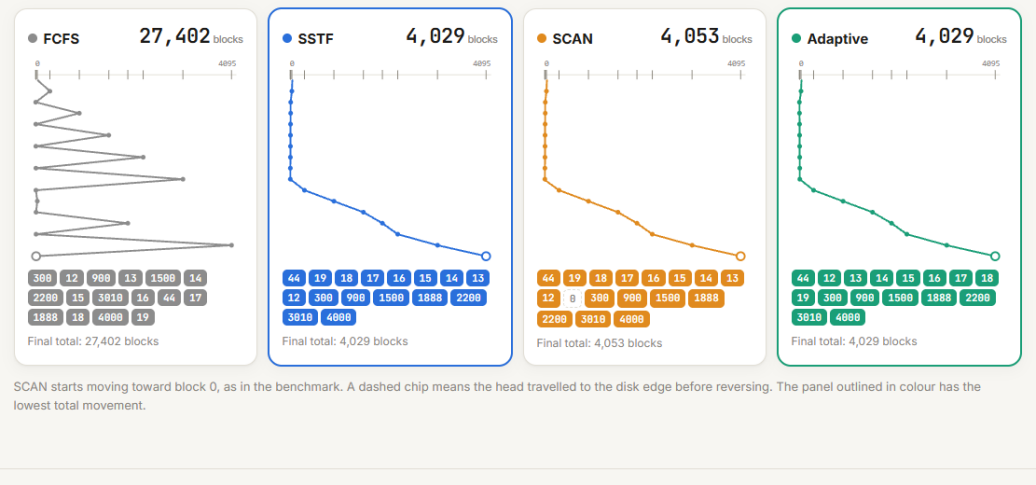


Fig. 28 Scheduler lab on the 16-request concurrent trace: FCFS zig-zags across the disk (27,402 blocks).

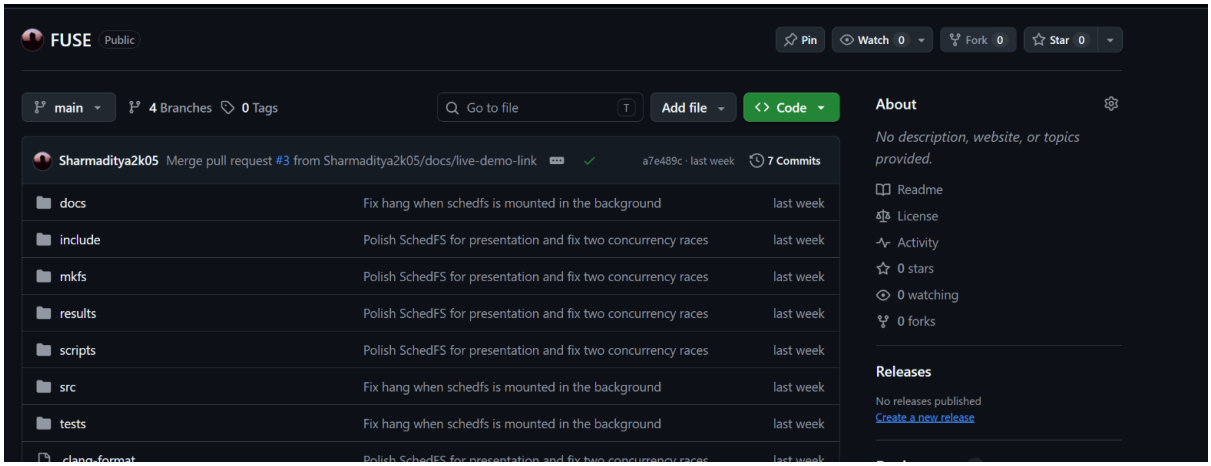


Fig. 29 Repository on GitHub, with the merged pull requests and the `github-pages` deployment.

7. TESTING DETAILS

7.1 Testing Methodology

Testing was carried out at the levels shown in Table 16. Each level adds a layer of the real system: the scheduler alone, then the core file-system API with threads but without FUSE, then the mounted file system driven by shell commands, and finally measurement.

Table 16 Testing levels and tools

Level	What is checked	Tool
Unit	exact service order and head movement of FCFS, SSTF, SCAN	`tests/test_scheduler.c`
Integration	create, write, read, truncate, sparse reads, persistence across remount, 4 concurrent writers	`tests/test_core.c`
System	a real FUSE mount under each policy, and in background mode, driven by `mkdir`, `cp`, `cmp`, `truncate`, `mv`, `rm`, `rmdir`	`tests/system.sh`
Functional	each functional requirement on a manually mounted image	shell session (Fig. 21–Fig. 24)
Boundary	name length limit, file larger than the direct blocks, invalid image sizes	shell session
Error handling	invalid options, missing or foreign image, non-empty directory, existing target	shell session (Fig. 34)
Concurrency	data races under 4 threads	ThreadSanitizer [13]
Performance	seek distance, waiting time and throughput per policy	`schedfs-bench`; mounted workload

Environment. Docker Desktop on Windows 11 running the `ubuntu:24.04` image, with GCC 13.3.0 and libfuse 3.14.0 (Fig. 19). The container had `/dev/fuse` and the `SYS_ADMIN` capability, and all tests ran as an unprivileged user, because `system.sh` refuses to run as root.

7.2 Test Cases

Table 17 lists 25 test cases. The “Actual output” column quotes the captured output.

Table 17 Test cases and results

ID	Objective	Module	Pre-condition	Input	Expected output	Actual output	Result
TC01	FCFS service order and movement	M6	none	head 53; textbook queue	order as queued; 640	assertion holds; “scheduler tests passed”	Pass
TC02	SSTF service order and movement	M6	none	same trace	65, 67, 37, 14, 98, 122, 124, 183; 236	assertion holds	Pass
TC03	SCAN service order and movement	M6	direction −1	same trace	37, 14, 65, …, 183; 236	assertion holds	Pass
TC04	Format a valid image	M1	no `disk.img`	`disk.img 16M`	4096 blocks, 256 inodes, data at 10	“Formatted disk.img: 4096 blocks, 256 inodes, data starts at block 10”	Pass
TC05	Never overwrite an image	M1	`disk.img` exists	same command	error, exit 1	“open (…): File exists”	Pass
TC06	Reject invalid sizes	M1	none	`10K`, `64K`	size error, exit 2	“size must be a positive multiple of 4096 (M/MiB/G/GiB accepted)”	Pass
TC07	Mount and report geometry	M2	formatted image	`--sched=sstf`	geometry printed; mount listed	“scheduler=sstf”; type `fuse.schedfs`	Pass
TC08	Create, write and read a file	M3	mounted	`echo`, `cat`	“Operating Systems”	“Operating Systems”	Pass
TC09	Large file through indirect block	M3, M4	mounted	100,000 random bytes	identical copy	`cmp`: “identical”; size 100000	Pass
TC10	Extend with `truncate`	M3, M4	18-byte file	`truncate -s 8192`	size 8192	“8192 bytes”	Pass
TC11	Rename and delete	M3	files exist	`mv`, `rm`	entries renamed, removed	listing shows only `demo.txt`	Pass
TC12	Name length limit (boundary)	M3	mounted	70-character name	`ENAMETOOLONG`	“File name too long”	Pass
TC13	Remove non-empty directory	M3	directory has files	`rmdir`	`ENOTEMPTY`	“Directory not empty”	Pass
TC14	Rename onto existing target	M3	both files exist	`mv a b`	refused (documented limit)	“cannot overwrite …: File exists”	Pass
TC15	Read missing file	M3	mounted	`cat missing.txt`	`ENOENT`	“No such file or directory”	Pass

continued on next page

ID	Objective	Module	Pre-condition	Input	Expected output	Actual output	Result
TC16	Invalid mount options	M2	none	`--sched=lifo`; `--alpha=-1`	usage; weight error	usage text; “alpha and beta must be non-negative”	Pass
TC17	Missing or foreign image	M2, M3	none	`nosuch.img`; 1 MiB of zeros	clear error, exit 1	“No such file or directory”; “Invalid argument”	Pass
TC18	Persistence across remount	M3, M7	data written; unmounted	remount with FCFS	same content	“Operating Systems”	Pass
TC19	Concurrent writers (integration)	M3–M7	fresh image	4 threads × 64 KiB	each file intact after remount	“core persistence and concurrency tests passed”	Pass
TC20	No data races	M3–M7	TSan build	`test_core`	no reports; exit 0	no reports; “exit status: 0”	Pass
TC21	Mounted tests, all policies	M1–M7	`/dev/fuse`	`system.sh`	all steps succeed	“FUSE system tests passed”	Pass
TC22	Background (daemon) mount	M2, M7	no `-f`	relative image path	I/O works; data persists	part of TC21; passed	Pass
TC23	Metrics and DEBUG log	M8	`--verbose`	20,000-byte write	unmount summary; allocations	“allocated block 13” … “125 requests, total seek 379 blocks”	Pass
TC24	Benchmark reproducibility	M6, M9	committed CSV	`schedfs-bench --all`	FCFS, SSTF, SCAN seek totals unchanged	all 15 equal; Adaptive differs in 3 of 5 (clock-dependent)	Pass
TC25	Policy effect on mounted FS	M5–M7	same image per run	4 writers + 4 readers, 3 trials	equal requests; SSTF < FCFS	4,927 requests each; SSTF −40 %	Pass

7.3 Testing Results

All 25 test cases passed. Table 18 groups them by level.

Table 18 Summary of test results

Level	Test cases	Executed	Passed	Failed
Unit	TC01–TC03	3	3	0
Functional and boundary	TC04–TC12, TC23	10	10	0
Error handling	TC13–TC17	5	5	0
Integration and concurrency	TC18–TC20	3	3	0
System	TC21–TC22	2	2	0
Performance	TC24–TC25	2	2	0
Total		**25**	**25**	**0**

Observations and limitations found during testing.

- `make test` prints "Ignoring invalid max threads value 4294967295 > max (100000)" once per foreground mount (Fig. 33). This message comes from libfuse 3.14.0, which reports an uninitialised default when `fuse_main()` runs a multi-threaded loop. SchedFS does not set this value, and the message does not affect any test.
- The formatter does not accept a `K` suffix (TC06). The message lists the accepted suffixes, but it could be clearer for a size such as `64K`, which is a multiple of 4096.
- `rename` onto an existing name is refused rather than replacing the target (TC14). This is a documented limitation, not a crash.
- Timing columns depend on the machine. On the Linux run, SSTF and SCAN have almost the same average wait on the concurrent trace (1.35 and 1.37 ms), whereas the repository's committed CSV, recorded on another machine, ranks SCAN clearly best. Only the seek columns of FCFS, SSTF and SCAN are exactly repeatable.

7.3.1 Benchmark results

Table 19 reports the benchmark run captured in Fig. 36. Fig. 30 and Fig. 31 plot the same data.

Table 19 Benchmark results (Linux run, Fig. 36); best seek per workload in bold

Workload	Policy	Total seek (blocks)	Avg. wait (ms)	Max. wait (ms)	Throughput (req/s)
Sequential (8)	FCFS	50	0.113	0.162	44,185
	SSTF	**43**	0.102	0.165	45,078
	SCAN	**43**	0.101	0.155	47,900
	Adaptive	50	0.113	0.162	44,185
Random (7)	FCFS	631	0.633	1.191	5,252
	SSTF	295	0.265	0.629	10,589
	SCAN	**229**	0.194	0.454	13,231
	Adaptive	295	0.371	0.629	10,586
Clustered (7)	FCFS	**58**	0.106	0.166	37,294
	SSTF	**58**	0.106	0.165	37,455
	SCAN	164	0.288	0.377	17,547
	Adaptive	**58**	0.109	0.170	36,459
Mixed (11)	FCFS	736	0.538	1.246	6,944
	SSTF	237	0.190	0.510	18,595
	SCAN	229	0.162	0.488	19,314
	Adaptive	**209**	0.164	0.448	20,776
Concurrent (16)	FCFS	27,402	18.923	46.986	291
	SSTF	**4,029**	1.355	6.216	1,946
	SCAN	4,053	1.373	6.265	1,935
	Adaptive	4,523	2.216	7.205	1,737

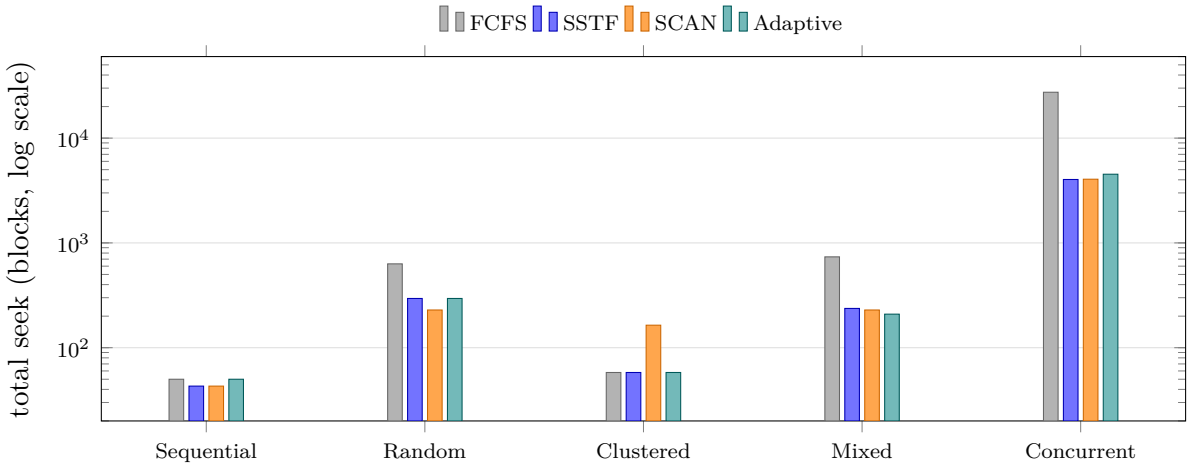


Fig. 30 Total seek distance per workload and policy (lower is better).

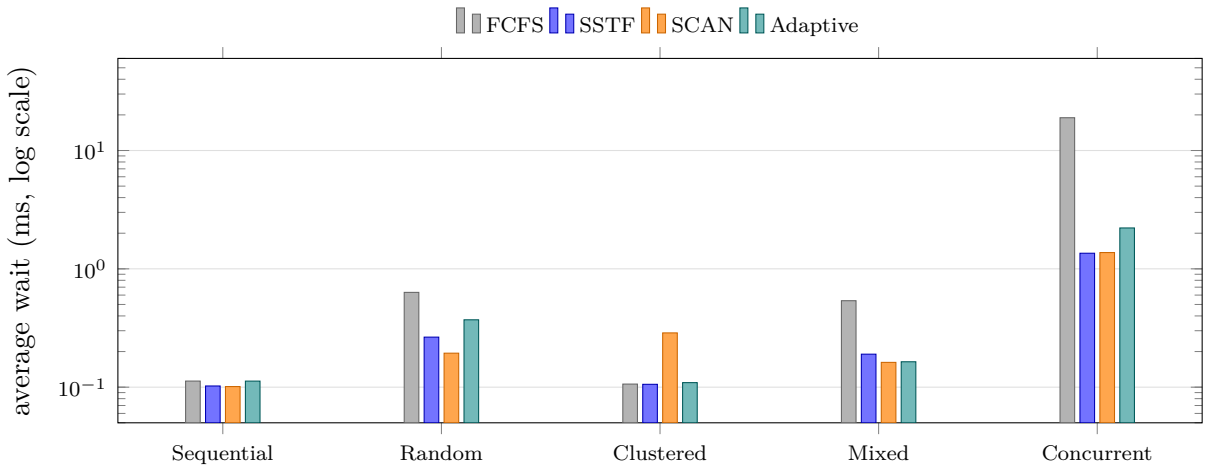


Fig. 31 Average waiting time per workload and policy (lower is better).

Analysis.

- *Contention favours position-aware policies.* On the concurrent trace FCFS moves the head 27,402 blocks against 4,029 for SSTF, which is 6.8 times as far, and its average wait is about 14 times longer (18.9 against 1.35 ms).
- *No policy wins everywhere.* SCAN is best on the random trace (229) but worst on the clustered trace (164 against 58), because it travels to the disk edge even when every request is nearby.
- *The Adaptive policy depends on timing.* On the mixed trace it beat every other policy in this run (209), while the committed CSV recorded 237. Its score uses measured waiting times, so small timing differences change its choices. This is the expected cost of ageing based on real time.

7.3.2 Scheduling on the mounted file system

The benchmark isolates the scheduler. To measure the policies on real file-system traffic, the same workload was run on a fresh copy of one image under each policy, three times each (Fig. 37). Four background `head` processes each wrote 256 KiB at the same time, four `cat` processes then read the files back, and the files were deleted. Every run served exactly 4,927 block requests, so only the *order* of service differed. Table 20 and Fig. 32 summarise the results.

Table 20 Total head movement on the mounted file system (4,927 requests per run)

Policy	Trial 1	Trial 2	Trial 3	Mean	Per request	vs. FCFS
FCFS	199,289	195,909	186,121	193,773	39.3	—
SSTF	117,041	112,565	118,935	**116,180**	**23.6**	−40.0 %
SCAN	126,521	134,431	136,659	132,537	26.9	−31.6 %
Adaptive	184,475	179,565	195,087	186,376	37.8	−3.8 %

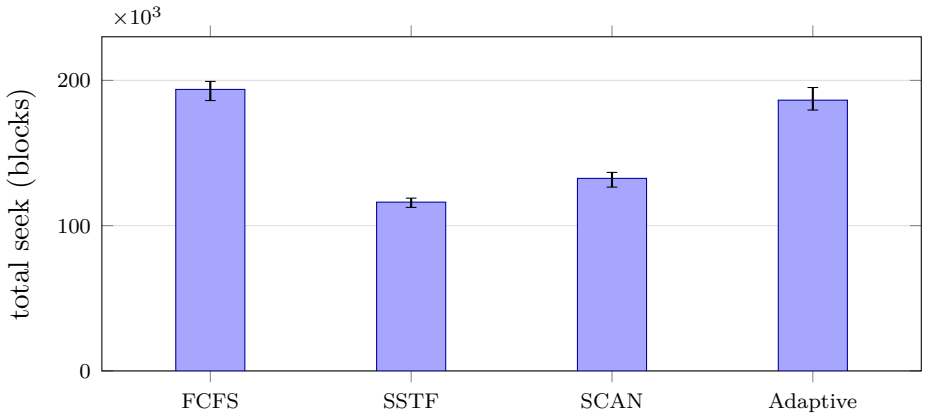


Fig. 32 Mean total seek on the mounted file system; error bars show the minimum and maximum of three trials.

The ranking on real traffic is SSTF, then SCAN, then Adaptive, then FCFS, and the spread between trials is small compared with the differences between policies. The Adaptive policy behaves almost like FCFS here, and its score explains why. The distance term is normalised by the disk size, 4,096 blocks. Requests on this small image are rarely more than a few hundred blocks apart, so $\alpha \cdot d/N$ stays below about 0.05, while $\beta \cdot w/w_{\max}$ ranges up to 0.3. The age term dominates, and the policy serves requests roughly in arrival order. Lowering β or normalising distance by the span of the pending requests would move it towards SSTF. This is a design improvement identified by testing.

7.4 Testing Screenshots

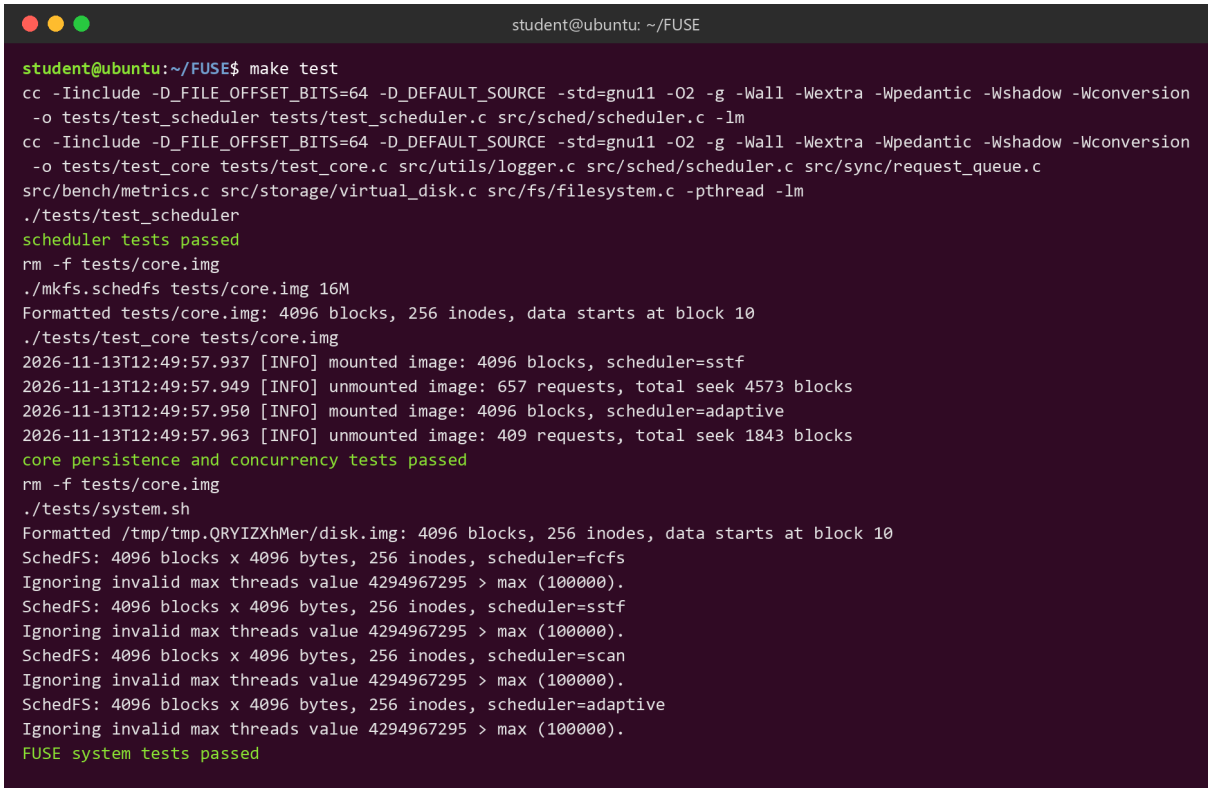


Fig. 33 `make test`: scheduler unit test, core persistence and concurrency test, and mounted system tests under all four policies, all passing (TC01–TC03, TC19, TC21, TC22).

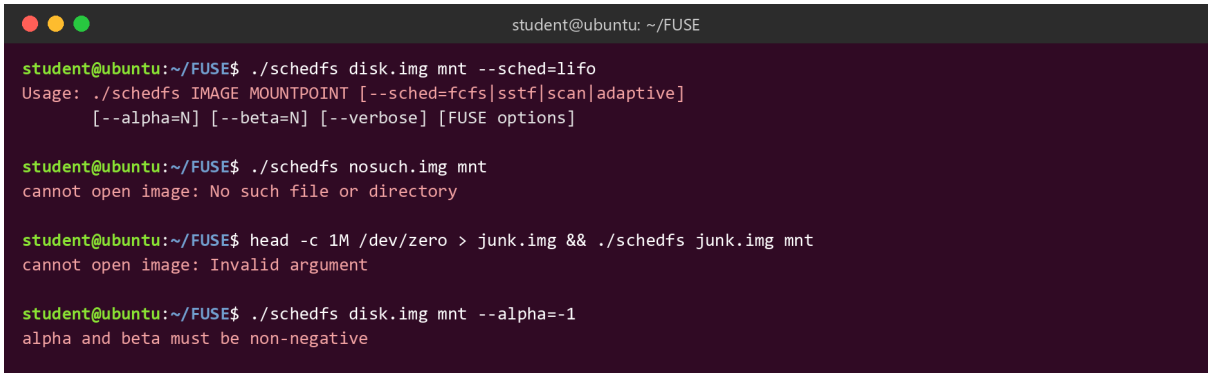


Fig. 34 Error handling at mount time: unknown policy, missing image, non-SchedFS image and negative weight (TC16, TC17).

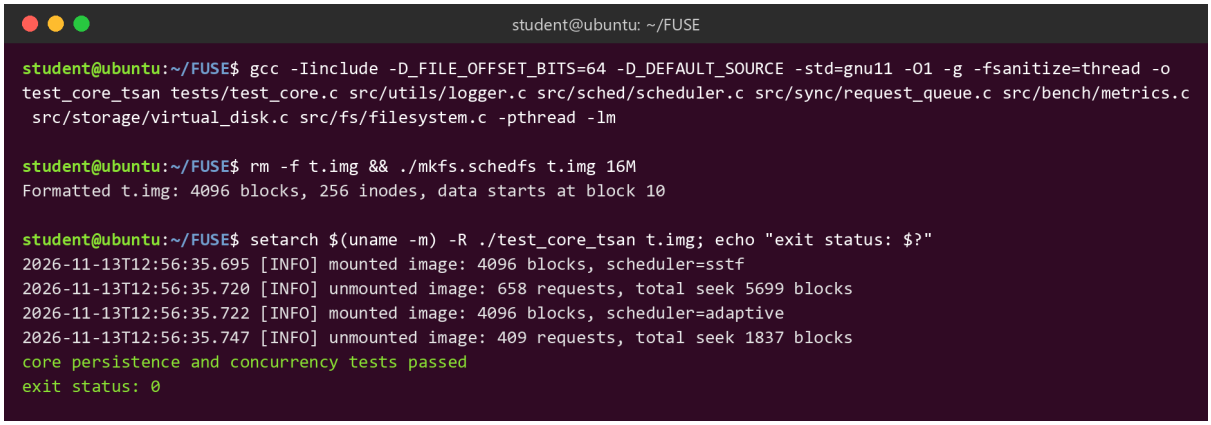


Fig. 35 ThreadSanitizer build of the core concurrency test: no data races reported, exit status 0 (TC20).

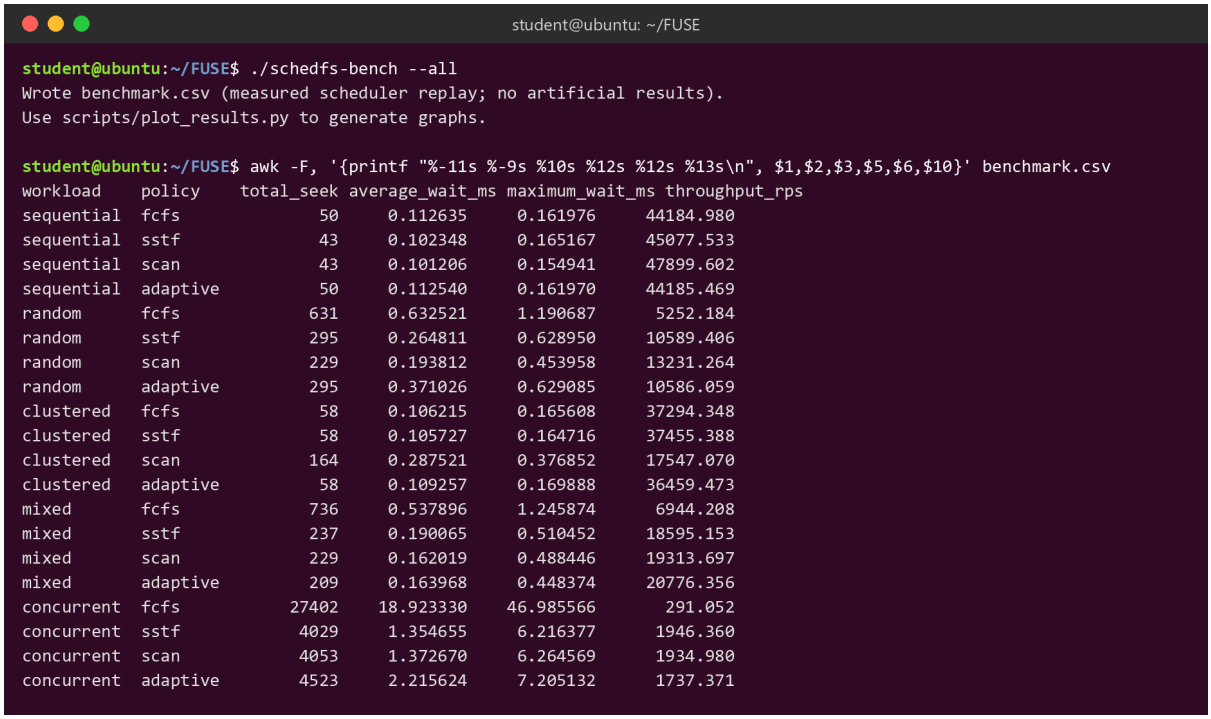


Fig. 36 Benchmark run and its CSV output (TC24; data in Table 19).

```
student@ubuntu: ~/FUSE

student@ubuntu:~/FUSE$ cat workload.sh
for i in 1 2 3 4; do head -c 262144 /dev/urandom > mnt/w$i.bin & done; wait
for i in 1 2 3 4; do cat mnt/w$i.bin > /dev/null & done; wait
rm mnt/w*.bin

student@ubuntu:~/FUSE$ for p in fcfs sstf scan adaptive; do for t in 1 2 3; do ... ; done; done
policy=fcfs      trial=1  4927 requests, total seek 199289 blocks
policy=fcfs      trial=2  4927 requests, total seek 195909 blocks
policy=fcfs      trial=3  4927 requests, total seek 186121 blocks
policy=sstf      trial=1  4927 requests, total seek 117041 blocks
policy=sstf      trial=2  4927 requests, total seek 112565 blocks
policy=sstf      trial=3  4927 requests, total seek 118935 blocks
policy=scan      trial=1  4927 requests, total seek 126521 blocks
policy=scan      trial=2  4927 requests, total seek 134431 blocks
policy=scan      trial=3  4927 requests, total seek 136659 blocks
policy=adaptive  trial=1  4927 requests, total seek 184475 blocks
policy=adaptive  trial=2  4927 requests, total seek 179565 blocks
policy=adaptive  trial=3  4927 requests, total seek 195087 blocks
```

Fig. 37 The same mounted workload under each policy, three trials each (TC25; data in Table 20).

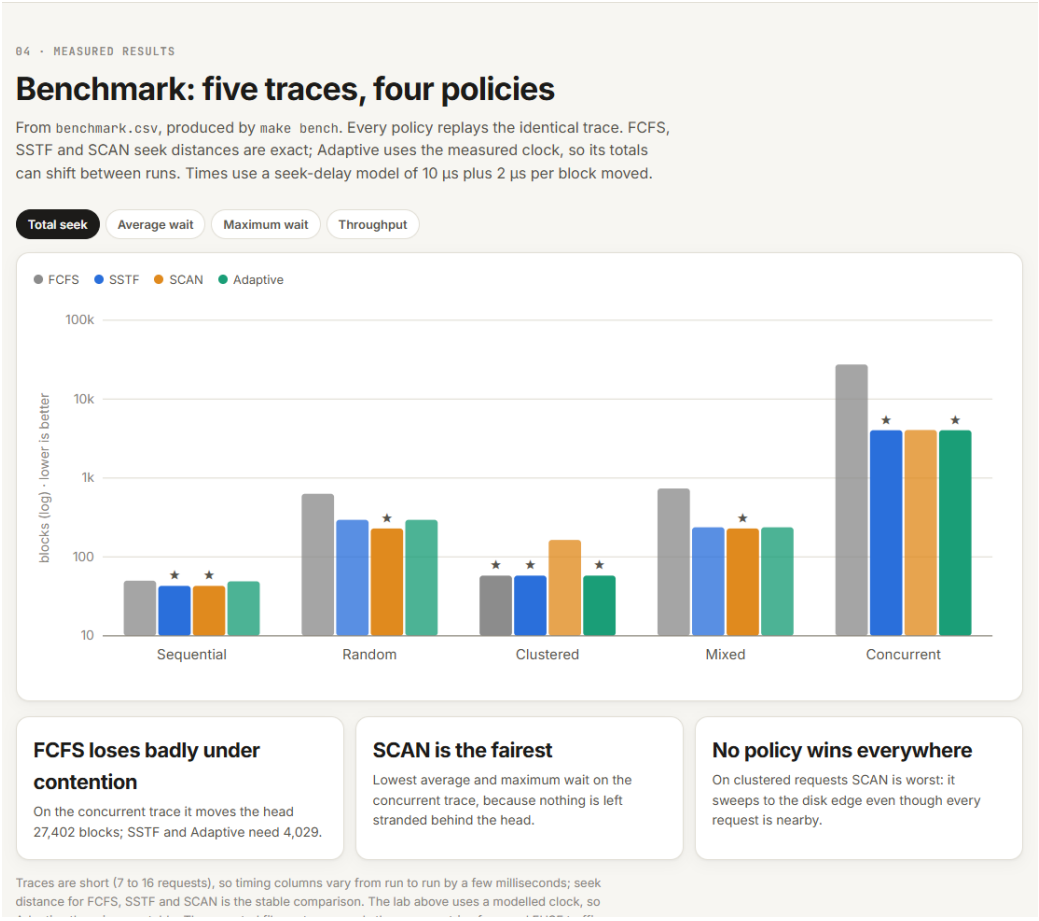


Fig. 38 Benchmark view of the demo page, drawn from the repository's committed `benchmark.csv`.

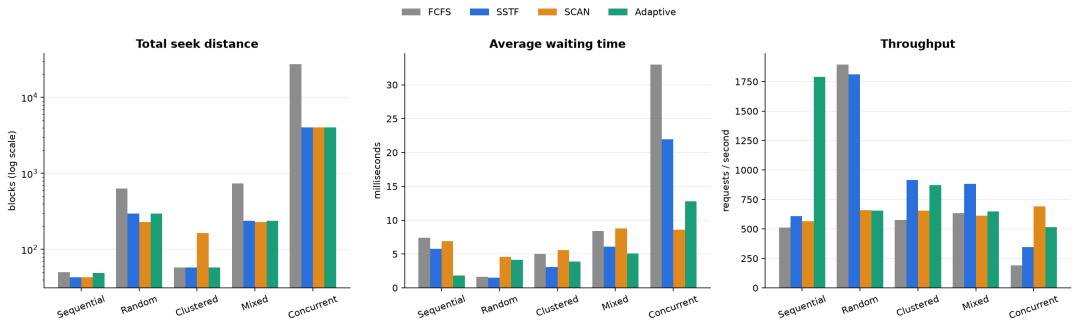


Fig. 39 Summary graph produced by `scripts/plot_results.py` from the repository's committed `benchmark.csv`.

Fig. 38 and Fig. 39 use the repository's committed CSV, recorded on a different machine. Their seek totals for FCFS, SSTF and SCAN equal those in Table 19, while their timing values and Adaptive totals differ, as discussed under TC24.

8. REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

[3] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 1st ed. Arpaci-Dusseau Books, 2018, [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[4] M. Kerrisk, *The Linux Programming Interface*. San Francisco, CA, USA: No Starch Press, 2010.

[5] B. L. Worthington, G. R. Ganger, and Y. N. Patt, "Scheduling algorithms for modern disk drives," in *Proc. ACM SIGMETRICS Conf. Measurement and Modeling of Computer Systems*, Nashville, TN, USA, 1994, pp. 241–251.

[6] ISO/IEC, "Information technology – programming languages – C," ISO/IEC 9899:2011, 2011.

[7] libfuse contributors, "libfuse: The reference implementation of the Linux FUSE (filesystem in userspace) interface," GitHub repository, [Online]. Available: https://github.com/libfuse/libfuse.

[8] libfuse Developers, "libfuse API documentation," libfuse project, [Online]. Available: https://libfuse.github.io/doxygen/.

[9] The Linux Kernel Documentation, "FUSE," The Linux Kernel documentation, Filesystems section, [Online]. Available: https://www.kernel.org/doc/html/latest/filesystems/fuse.html.

[10] IEEE and The Open Group, "The open group base specifications issue 7 (IEEE std 1003.1-2017)," IEEE Std 1003.1-2017, 2018, [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/.

[11] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy 2010)*, Austin, TX, USA, 2010, pp. 56–61.

[12] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[13] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data race detection in practice," in *Proc. Workshop on Binary Instrumentation and Applications (WBIA '09)*, New York, NY, USA, 2009, pp. 62–71.

[14] B. K. R. Vangoor, V. Tarasov, and E. Zadok, "To FUSE or not to FUSE: Performance of user-space file systems," in *Proc. 15th USENIX Conf. File and Storage Technologies (FAST '17)*, Santa Clara, CA, USA, 2017, pp. 59–72.

[15] R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.

[16] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2005.

[17] M. J. Bach, *The Design of the UNIX Operating System*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1986.

[18] M. K. McKusick, W. N. Joy, S. J. Leffler, and R. S. Fabry, "A fast file system for UNIX," *ACM Transactions on Computer Systems*, vol. 2, no. 3, pp. 181–197, 1984.

[19] T. J. Teorey and T. B. Pinkerton, "A comparative analysis of disk scheduling policies," *Communications of the ACM*, vol. 15, no. 3, pp. 177–184, 1972.

[20] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Boston, MA, USA: Pearson, 2018.

[21] B. Lewis and D. J. Berg, *Pthreads Primer: A Guide to Multithreaded Programming*. Upper Saddle River, NJ, USA: Prentice Hall, 1996.

[22] M. Kerrisk, "pthread_cond_wait(3p) — Linux manual page," Linux man-pages project, [Online]. Available: https://man7.org/linux/man-pages/man3/pthread_cond_wait.3p.html.

A. SOURCE LISTINGS OF THE SCHEDULING PATH

The three files below form the complete path from a submitted request to the disk: the queue, the scheduler and the dispatcher. They are reproduced in full from the repository.

Listing 14 `src/sync/request_queue.c` (complete)

```
/*
 * request_queue.c
 *
 * Request lifecycle and the bounded producer-consumer queue (mutex + condvars).
 */
#include "request_queue.h"
#include <errno.h>
#include <stdatomic.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

static atomic_uint_fast64_t next_id = 1;

int request_init(disk_request_t *r, rq_op_t op, uint32_t block, void *buffer) {
    memset(r, 0, sizeof(*r));
    r->id = atomic_fetch_add(&next_id, 1);
    r->op = op;
    r->block = block;
    r->buffer = buffer;
    r->producer = pthread_self();
    clock_gettime(CLOCK_MONOTONIC, &r->arrival);
    int e = pthread_mutex_init(&r->mutex, NULL);
    if (e)
        return -e;
    e = pthread_cond_init(&r->completed_cv, NULL);
    if (e) {
        pthread_mutex_destroy(&r->mutex);
        return -e;
    }
    return 0;
}

void request_destroy(disk_request_t *r) {
    pthread_cond_destroy(&r->completed_cv);
    pthread_mutex_destroy(&r->mutex);
}

int request_wait(disk_request_t *r) {
    pthread_mutex_lock(&r->mutex);
    while (!r->completed)
        pthread_cond_wait(&r->completed_cv, &r->mutex);
    int result = r->error ? -r->error : (r->result < 0 ? -EIO : 0);
    pthread_mutex_unlock(&r->mutex);
    return result;
}

int rq_init(request_queue_t *q, size_t capacity) {
    memset(q, 0, sizeof(*q));
    if (!capacity)
```

```
51          return -EINVAL;
52      q->items = calloc(capacity, sizeof(*q->items));
53      if (!q->items)
54          return -ENOMEM;
55      q->capacity = capacity;
56      pthread_mutex_init(&q->mutex, NULL);
57      pthread_cond_init(&q->not_empty, NULL);
58      pthread_cond_init(&q->not_full, NULL);
59      return 0;
60  }
61
62  int rq_submit(request_queue_t *q, disk_request_t *r) {
63      pthread_mutex_lock(&q->mutex);
64      while (q->count == q->capacity && !q->stopping)
65          pthread_cond_wait(&q->not_full, &q->mutex);
66      if (q->stopping) {
67          pthread_mutex_unlock(&q->mutex);
68          return -ESHUTDOWN;
69      }
70      q->items[q->count++] = r;
71      pthread_cond_signal(&q->not_empty);
72      pthread_mutex_unlock(&q->mutex);
73      return 0;
74  }
75
76  disk_request_t *rq_take(request_queue_t *q, const scheduler_t *scheduler) {
77      pthread_mutex_lock(&q->mutex);
78      while (!q->count && !q->stopping)
79          pthread_cond_wait(&q->not_empty, &q->mutex);
80      if (!q->count) {
81          pthread_mutex_unlock(&q->mutex);
82          return NULL;
83      }
84      struct timespec now;
85      clock_gettime(CLOCK_MONOTONIC, &now);
86      size_t index = scheduler_pick(scheduler, q->items, q->count, now);
87      disk_request_t *r = q->items[index];
88      memmove(&q->items[index], &q->items[index + 1], (q->count - index - 1) *
        ↪ sizeof(*q->items));
89      --q->count;
90      pthread_cond_signal(&q->not_full);
91      pthread_mutex_unlock(&q->mutex);
92      return r;
93  }
94
95  void rq_stop(request_queue_t *q) {
96      pthread_mutex_lock(&q->mutex);
97      q->stopping = true;
98      pthread_cond_broadcast(&q->not_empty);
99      pthread_cond_broadcast(&q->not_full);
100     pthread_mutex_unlock(&q->mutex);
101 }
102
103 void rq_destroy(request_queue_t *q) {
104     free(q->items);
105     pthread_cond_destroy(&q->not_empty);
106     pthread_cond_destroy(&q->not_full);
107     pthread_mutex_destroy(&q->mutex);
```

```
}
```

Listing 15 `src/sched/scheduler.c` (complete)

```
/*
 * scheduler.c
 *
 * Request selection for each policy, with deterministic (arrival, id) tie-
    ↪ breaking.
 */
#include "scheduler.h"
#include <math.h>
#include <string.h>

static double elapsed_ms(struct timespec a, struct timespec b) {
    return (double)(b.tv_sec - a.tv_sec) * 1000.0 + (double)(b.tv_nsec - a.
    ↪ tv_nsec) / 1000000.0;
}

int scheduler_parse(const char *name, sched_policy_t *out) {
    if (!strcmp(name, "fcfs"))
        *out = SCHED_FCFS;
    else if (!strcmp(name, "sstf"))
        *out = SCHED_SSTF;
    else if (!strcmp(name, "scan"))
        *out = SCHED_SCAN;
    else if (!strcmp(name, "adaptive"))
        *out = SCHED_ADAPTIVE;
    else
        return -1;
    return 0;
}

const char *scheduler_name(sched_policy_t p) {
    static const char *names[] = {"fcfs", "sstf", "scan", "adaptive"};
    return p <= SCHED_ADAPTIVE ? names[p] : "unknown";
}

static uint32_t distance(uint32_t a, uint32_t b) {
    return a > b ? a - b : b - a;
}

static int older(const disk_request_t *a, const disk_request_t *b) {
    if (a->arrival.tv_sec != b->arrival.tv_sec)
        return a->arrival.tv_sec < b->arrival.tv_sec;
    if (a->arrival.tv_nsec != b->arrival.tv_nsec)
        return a->arrival.tv_nsec < b->arrival.tv_nsec;
    return a->id < b->id;
}

size_t scheduler_pick(const scheduler_t *s, disk_request_t *const *items, size_t
    ↪ count,
                      struct timespec now) {
    size_t best = 0;
    if (count < 2 || s->policy == SCHED_FCFS) {
        for (size_t i = 1; i < count; ++i)
            if (older(items[i], items[best]))
                best = i;
```

```
        return best;
    }
    if (s->policy == SCHED_SSTF) {
        for (size_t i = 1; i < count; ++i) {
            uint32_t di = distance(s->head, items[i]->block);
            uint32_t db = distance(s->head, items[best]->block);
            if (di < db || (di == db && older(items[i], items[best])))
                best = i;
        }
        return best;
    }
    if (s->policy == SCHED_SCAN) {
        bool found = false;
        for (size_t i = 0; i < count; ++i) {
            bool direction_ok =
                s->scan_direction > 0 ? items[i]->block >= s->head : items[i]->
    ↪ block <= s->head;
            if (!direction_ok)
                continue;
            if (!found ||
                distance(s->head, items[i]->block) < distance(s->head, items[best
    ↪ ]->block) ||
                (distance(s->head, items[i]->block) == distance(s->head, items[
    ↪ best]->block) &&
                 older(items[i], items[best]))) {
                best = i;
                found = true;
            }
        }
        if (found)
            return best;
        for (size_t i = 1; i < count; ++i) {
            if (distance(s->head, items[i]->block) < distance(s->head, items[best
    ↪ ]->block) ||
                (distance(s->head, items[i]->block) == distance(s->head, items[
    ↪ best]->block) &&
                 older(items[i], items[best])))
                best = i;
        }
        return best;
    }
    double max_wait = 0.0;
    for (size_t i = 0; i < count; ++i) {
        double w = elapsed_ms(items[i]->arrival, now);
        if (w > max_wait)
            max_wait = w;
    }
    double best_score = INFINITY;
    for (size_t i = 0; i < count; ++i) {
        double dn = (double)distance(s->head, items[i]->block) / (double)s->
    ↪ total_blocks;
        double wn = max_wait > 0.0 ? elapsed_ms(items[i]->arrival, now) /
    ↪ max_wait : 0.0;
        double score = s->alpha * dn - s->beta * wn;
        if (score < best_score - 1e-12 ||
            (fabs(score - best_score) <= 1e-12 && older(items[i], items[best]))) {
    ↪
            best = i;
```

```
            best_score = score;
        }
    }
    return best;
}
```

Listing 16 `src/storage/virtual_disk.c` (complete)

```
/*
 * virtual_disk.c
 *
 * Image-file block device. Only the dispatcher thread calls pread/pwrite.
 */
#include "virtual_disk.h"
#include "layout.h"
#include "logger.h"
#include <errno.h>
#include <fcntl.h>
#include <string.h>
#include <unistd.h>

static double diff_ms(struct timespec a, struct timespec b) {
    return (double)(b.tv_sec - a.tv_sec) * 1000.0 + (double)(b.tv_nsec - a.tv_nsec) / 1000000.0;
}

static ssize_t full_pio(int fd, void *buf, size_t n, off_t off, bool write_op) {
    size_t done = 0;
    while (done < n) {
        ssize_t rc = write_op ? pwrite(fd, (char *)buf + done, n - done, off + (off_t)done)
                              : pread(fd, (char *)buf + done, n - done, off + (off_t)done);
        if (rc < 0 && errno == EINTR)
            continue;
        if (rc <= 0)
            return rc < 0 ? -errno : -EIO;
        done += (size_t)rc;
    }
    return (ssize_t)done;
}

static void *dispatcher(void *arg) {
    virtual_disk_t *d = arg;
    while (true) {
        disk_request_t *r = rq_take(&d->queue, &d->scheduler);
        if (!r)
            break;
        clock_gettime(CLOCK_MONOTONIC, &r->service_start);
        uint32_t from = d->scheduler.head;
        r->result = full_pio(d->fd, r->buffer, SFS_BLOCK_SIZE, (off_t)r->block * SFS_BLOCK_SIZE,
                             r->op == RQ_WRITE);
        r->error = r->result < 0 ? (int)-r->result : 0;
        clock_gettime(CLOCK_MONOTONIC, &r->completion);
        metrics_record(&d->metrics, from, r->block, diff_ms(r->arrival, r->service_start));
        if (d->scheduler.policy == SCHED_SCAN) {
```

```
            if (d->scheduler.scan_direction > 0 && r->block < from)
                d->scheduler.scan_direction = -1;
            else if (d->scheduler.scan_direction < 0 && r->block > from)
                d->scheduler.scan_direction = 1;
        }
        d->scheduler.head = r->block;
        pthread_mutex_lock(&r->mutex);
        r->completed = true;
        pthread_cond_signal(&r->completed_cv);
        pthread_mutex_unlock(&r->mutex);
    }
    return NULL;
}

int vdisk_open(virtual_disk_t *d, const char *path, uint32_t blocks,
    ↪ sched_policy_t policy,
               double alpha, double beta, size_t capacity) {
    memset(d, 0, sizeof(*d));
    d->fd = open(path, O_RDWR | O_CLOEXEC);
    if (d->fd < 0)
        return -errno;
    d->blocks = blocks;
    d->scheduler = (scheduler_t){policy, blocks, 0, 1, alpha, beta};
    int rc = rq_init(&d->queue, capacity);
    if (rc) {
        close(d->fd);
        return rc;
    }
    metrics_init(&d->metrics);
    rc = pthread_create(&d->thread, NULL, dispatcher, d);
    if (rc) {
        rq_destroy(&d->queue);
        close(d->fd);
        return -rc;
    }
    d->running = 1;
    return 0;
}

static int transfer(virtual_disk_t *d, uint32_t block, void *buf, rq_op_t op) {
    if (block >= d->blocks)
        return -EINVAL;
    disk_request_t r;
    int rc = request_init(&r, op, block, buf);
    if (rc)
        return rc;
    rc = rq_submit(&d->queue, &r);
    if (!rc)
        rc = request_wait(&r);
    request_destroy(&r);
    return rc;
}

int vdisk_read(virtual_disk_t *d, uint32_t b, void *buf) {
    return transfer(d, b, buf, RQ_READ);
}

int vdisk_write(virtual_disk_t *d, uint32_t b, const void *buf) {
```

```
103     return transfer(d, b, (void *)buf, RQ_WRITE);
104 }
105
106 void vdisk_close(virtual_disk_t *d) {
107     if (!d->running)
108         return;
109     rq_stop(&d->queue);
110     pthread_join(d->thread, NULL);
111     metrics_finish(&d->metrics);
112     fsync(d->fd);
113     close(d->fd);
114     rq_destroy(&d->queue);
115     d->running = 0;
116 }
```

B. REPRODUCING THE RESULTS

On Ubuntu 22.04 or 24.04 with `/dev/fuse` available, the following commands reproduce every result in this report. Run them as a normal user, not as root.

Listing 17 Commands used to produce the results

```
sudo apt install -y build-essential pkg-config libfuse3-dev fuse3 \
                    python3-pandas python3-matplotlib
git clone https://github.com/Sharmaditya2k05/FUSE.git && cd FUSE
make                                  # Figure: build
./mkfs.schedfs disk.img 16M           # TC04
mkdir -p mnt && ./schedfs disk.img mnt --sched=sstf
echo 'Operating Systems' > mnt/demo.txt && cat mnt/demo.txt
fusermount3 -u mnt && cat filesystem.log
make test                             # TC01-TC03, TC19, TC21, TC22
./schedfs-bench --all                 # TC24, writes benchmark.csv
python3 scripts/plot_results.py       # graphs in results/
```